MODULE 04

File Operations in Python

Text and binary files, context managers, paths, JSON, CSV and XML

```
>>> from pathlib import Path
```

AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering



Learning Objectives

By the end of this module, you will be able to:

1. Read and write text and binary files with `open`, `with` and an explicit `encoding="utf-8"`
2. Implement context managers with `__enter__`/`__exit__` and `contextlib.contextmanager`
3. Manipulate paths, folders and files with `pathlib`, `os` and `shutil`
4. Serialize Python objects to JSON and CSV and read them back, including custom types
5. Explain the security risk of `pickle` and choose a suitable data format
6. Create well-formed XML documents with elements, attributes, entities and namespaces
7. Validate XML documents against a DTD and an XML Schema (XSD)
8. Parse XML in Python with SAX, DOM and ElementTree, and defend against XML attacks

MODULE 04 · FILE OPERATIONS

Module Roadmap

01 **Files and Text I/O**
Opening, reading and writing files safely — and the context managers behind with

02 **Paths and the File System**
pathlib, os, shutil and binary data

03 **Serializing Data With JSON**
Turning Python objects into portable, human-readable text — and back

04 **CSV Files and Other Formats**
Tabular data with the csv module, a word on pickle, and choosing a format

05 **XML Fundamentals**
Structure, syntax rules and namespaces of the Extensible Markup Language

06 **Validating XML with a DTD**
Document Type Definitions: grammar rules for elements, attributes and entities

07 **Validating XML with XML Schema**
XSD: typed, namespace-aware schemas written in XML

08 **Parsing XML in Python**
SAX, DOM, ElementTree and lxml — and parsing untrusted XML safely

01

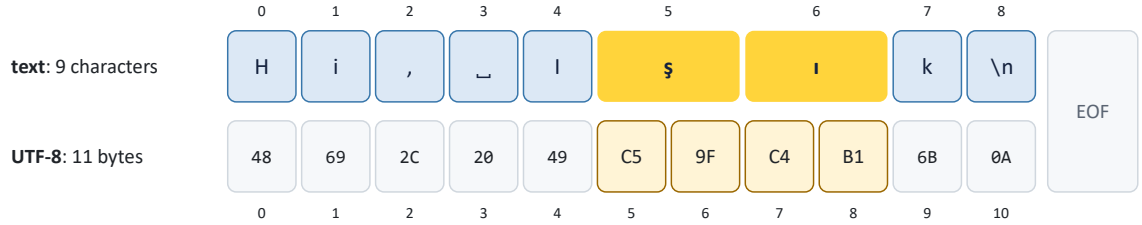
Files and Text I/O

Opening, reading and writing files safely — and the context managers behind with

- Files, streams and the layers of a file object
- Open modes, reading patterns, encodings
- Context managers: with, __enter__ / __exit__, @contextmanager
- Safe updates and file exceptions

Files, Characters and Bytes

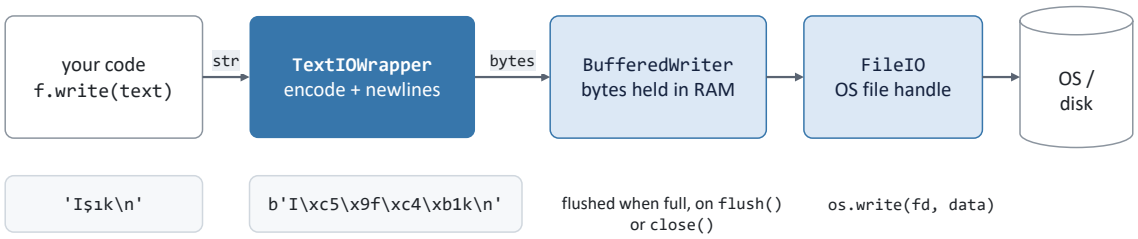
- A **text file** is a sequence of *characters*; a **binary file** (image, video, archive) is a sequence of *bytes*
- Positions start at 0: a file of *n* bytes ends at position *n* − 1
- The OS tracks the **end of file** (EOF); at EOF `read()` returns `""` or `b""` — no exception



Key idea

Characters ≠ bytes: `len(text)` counts characters, the file size counts bytes. Here the text `"Hi, Işık\n"` has 9 characters but occupies 11 bytes.

From Your Code to the Disk




```python
with open("layers.txt", "w", encoding="utf-8") as f:
    print(type(f).__name__, f.encoding)
    print(type(f.buffer).__name__)
    print(type(f.buffer.raw).__name__)
```

OUTPUT

```
TextIOWrapper utf-8
BufferedWriter
FileIO
```

Key idea

Written data may sit in the buffer until it is **flushed**. Closing the file flushes it — and `with` always closes.

Standard Streams

Stream	Connected to (default)	Used by
`sys.stdin`	keyboard	`input()`
`sys.stdout`	terminal	`print()`
`sys.stderr`	terminal	tracebacks, warnings, `logging`

report.py

```python
import sys

print("report ready")
print("3 rows skipped", file=sys.stderr)
sys.stdout.write("done\n")
```

TERMINAL

```
$ python report.py > report.txt
3 rows skipped
$ cat report.txt
report ready
done
```

★ **Key idea**

The standard streams are ordinary text file objects — `sys.stdout.write(s)` works like `f.write(s)` — and the shell can redirect each of them separately.

File Open Modes

`open(file, mode="r", encoding=None, newline=None)` returns a **file object**

Mode	Purpose	If the file exists	If it is missing
`"r"`	read (default)	read from the start	`FileNotFoundError`
`"w"`	write	**emptied** first	created
`"a"`	append	write at the end	created
`"x"`	exclusive create	`FileExistsError`	created
`"r+"`	read and write	kept; position 0	`FileNotFoundError`
`"w+"` / `"a+"`	read and write	emptied / write at end	created

Add `b` for binary data (`"rb"`, `"wb"`); `t` (text) is the default

! **Watch out**

`"w"` empties an existing file the moment it is opened — even if you never write.

Writing a Text File

write_accounts.py

```
with open("accounts.txt", "w", encoding="utf-8") as accounts:
    accounts.write("100,Jack Bauer,10000.0\n")
    accounts.write("200,Kate Austen,20000.0\n")
    print("300,James Sawyer,30000.0", file=accounts)
    accounts.writelines(["400,Ben Linus,40000.0\n",
                         "500,Sun Kwon,50000.0\n"])

with open("accounts.txt", encoding="utf-8") as accounts:
    print(accounts.read(), end="")
```

OUTPUT

```
100,Jack Bauer,10000.0
200,Kate Austen,20000.0
300,James Sawyer,30000.0
400,Ben Linus,40000.0
500,Sun Kwon,50000.0
```

- `write(s)` adds **no** newline — write `\n` yourself
- `print(..., file=f)` adds it for you
- `writelines` writes an iterable of strings
- Relative paths start from the **current working directory**

Reading a Text File Line by Line

read_accounts.py

```
with open("accounts.txt", encoding="utf-8") as accounts:
    print(f"{'Account':<10}{'Full Name':<20}{'Balance':>10}")
    for record in accounts:
        account, name, balance = record.rstrip("\n").split(",")
        print(f"{account:<10}{name:<20}{balance:>10}")
```

OUTPUT

```
Account   Full Name             Balance
100       Jack Bauer            10000.0
200       Kate Austen           20000.0
300       James Sawyer          30000.0
```

Key idea

A file object is an **iterator** over its lines (Module 2): lazy, one line in memory at a time.

Watch out

Each line keeps its trailing `\n` — strip it before splitting.

Reading Patterns Compared

Pattern	Returns	Memory	Best for
`for line in f:`	one line per iteration	one line	default choice, any file size
`f.read()`	whole file as one `str`	whole file	small files, full-text search
`f.readlines()`	`list` of all lines	whole file	small files, indexing lines
`f.readline()`	next line; `""` at EOF	one line	reading a header first
`f.read(n)`	up to *n* characters	*n* characters	fixed-size chunks

```python
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("alpha\nbeta\ngamma\n")

with open("notes.txt", encoding="utf-8") as f:
    header = f.readline()
    rest = f.readlines()
print(repr(header), rest)
```

OUTPUT

```
'alpha\n' ['beta\n', 'gamma\n']
```

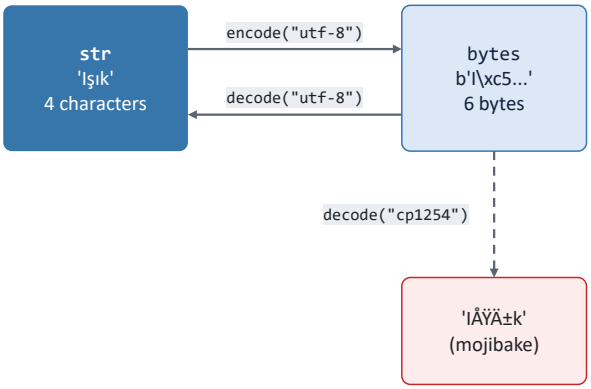
Tip

Iterate over the file object by default; `read()` and `readlines()` load the whole file into memory.

Characters, Bytes and Encodings

- An **encoding** maps characters to bytes; **UTF-8** uses 1–4 bytes per character
- Text mode *encodes* `str` → `bytes` on write and *decodes* `bytes` → `str` on read
- Decoding with the wrong encoding gives **mojibake** or a `UnicodeDecodeError`

```
>>> text = "Işık"
>>> data = text.encode("utf-8")
>>> data
b'I\xc5\x9f\xc4\xb1k'
>>> len(text), len(data)
(4, 6)
>>> data.decode("utf-8")
'Işık'
>>> data.decode("cp1254")
'IÅŸÄ±k'
```



Always Specify the Encoding

```
text = "Çağrı: 250 ₺"
with open("receipt.txt", "w", encoding="utf-8") as f:
    f.write(text)

with open("receipt.txt", "rb") as f:
    print(f.read())

try:
    with open("receipt.txt", encoding="ascii") as f:
        f.read()
except UnicodeDecodeError as err:
    print(err.reason, "at byte", err.start)
```

OUTPUT

```
b'\xc3\x87a\xc4\x9fr\xc4\xb1: 250 \xe2\x82\xba'
ordinal not in range(128) at byte 0
```

- Without `encoding=`, `open` uses the platform default — `cp1254` on Turkish Windows, UTF-8 on Linux/macOS
- Pass `encoding="utf-8"` on **every** text-mode `open`
- `"utf-8-sig"` skips the byte-order mark written by Excel

Python version note

PEP 686 makes UTF-8 mode the default in Python 3.15. Until then, be explicit; `-X warn_default_encoding` flags calls without `encoding=`.

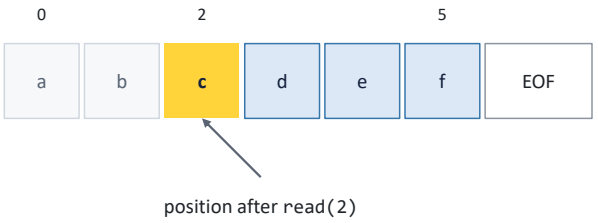
The File Position: `tell` and `seek`

```
with open("letters.txt", "w", encoding="utf-8") as f:
    f.write("abcdef")

with open("letters.txt", encoding="utf-8") as f:
    print(f.read(2))
    print(f.tell())
    print(f.read())
    print(repr(f.read()))
    f.seek(0)
    print(f.read(3))
```

OUTPUT

```
ab
2
cdef
''
abc
```



- The **file position** is where the next read or write happens
- `seek(0)` rewinds without reopening the file
- Text mode: seek only to 0 or to a value from `tell()`

QUIZ

Quick Check: Open Modes

You must create `report.txt`, but an existing report must **never** be overwritten. Which mode do you pass to `open`?

A `"w"`

B `"a"`

C `"x"`

D `"r+"`

ANSWER

Quick Check: Open Modes

You must create `report.txt`, but an existing report must **never** be overwritten. Which mode do you pass to `open`?

A `"w"`

B `"a"`

C `"x"`

D `"r+"`

Why

`"x"` creates the file and raises `FileExistsError` if it already exists. `"w"` would empty it, `"a"` would append to it and `"r+"` requires it to exist.

Why `with`? Releasing Resources

- Files, sockets, locks and database connections are **resources**: acquire → use → release
- An unclosed file may lose buffered data, stay locked (Windows) or exhaust OS handles
- `with` releases the resource however the block ends: normally, by `return`, or by an exception

Manual clean-up

```
log = open("run.log", "w", encoding="utf-8")
try:
    log.write("started\n")
finally:
    log.close()
```

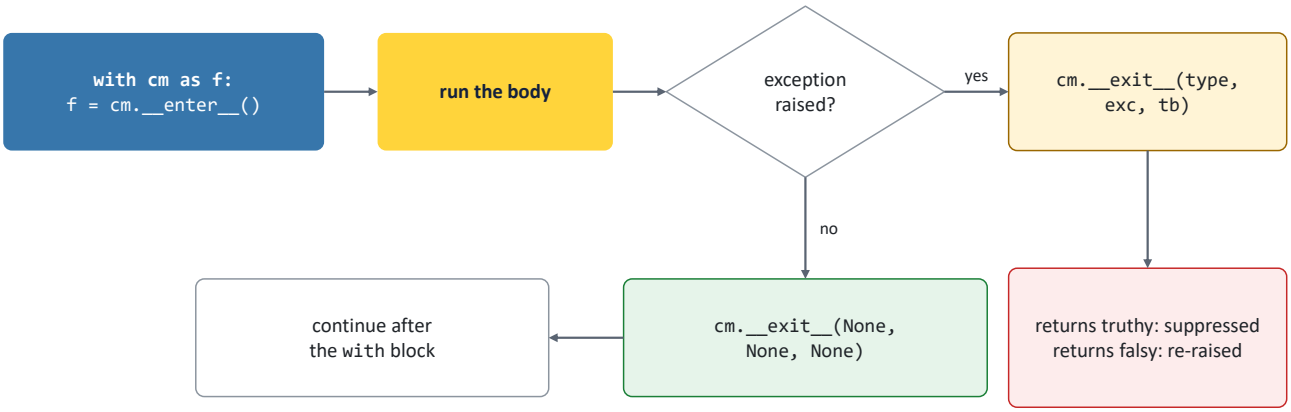
With a context manager

```
with open("run.log", "w", encoding="utf-8") as log:
    log.write("started\n")
print(log.closed)
```

OUTPUT

```
True
```


How `with` Works: The Protocol



Key idea

A **context manager** is any object with `__enter__` and `__exit__`. `__exit__` runs however the body ends; its return value decides whether an exception is swallowed.

Writing a Context Manager Class

tag.py

```python
class Tag:
    """Print an opening and a closing tag around a block."""

    def __init__(self, name: str) -> None:
        self.name = name

    def __enter__(self) -> "Tag":
        print(f"<{self.name}>")
        return self

    def __exit__(self, exc_type, exc, tb) -> bool:
        print(f"</{self.name}>")
        return False  # never suppress exceptions


with Tag("ul"):
    with Tag("li"):
        print("Python")
```

OUTPUT

```
<ul>
<li>
Python
</li>
</ul>
```

- `__enter__`'s return value is bound by `as`
- `__exit__` always runs; without an exception its three arguments are `None`
- Nested managers exit in reverse order

TRACE THE CODE

Exceptions Inside a `with` Block

```python
class Guard:
    def __enter__(self):
        print("enter")
        return self

    def __exit__(self, exc_type, exc, tb):
        print("exit", exc_type.__name__)
        return exc_type is ValueError


for error in (ValueError, KeyError):
    try:
        with Guard():
            raise error("oops")
        print("after with")
    except KeyError:
        print("handled outside")
```

ANSWER

Exceptions Inside a `with` Block

OUTPUT

```
enter
exit ValueError
after with
enter
exit KeyError
handled outside
```

- `__exit__` runs in both iterations, **before** any `except` clause
- For `ValueError` it returns `True`: the exception is suppressed, execution continues
- For `KeyError` it returns `False`: the exception propagates to the outer `try`

1 · FILES AND TEXT I/O

Generator-Based Context Managers

```python
from collections.abc import Iterator
from contextlib import contextmanager


@contextmanager
def tag(name: str) -> Iterator[None]:
    print(f"<{name}>")       # like __enter__
    try:
        yield                # the with body runs here
    finally:
        print(f"</{name}>")  # like __exit__


with tag("ul"), tag("li"):
    print("Python")
```

OUTPUT

```
<ul>
<li>
Python
</li>
</ul>
```

In the generator	Role in `with`
code before `yield`	`__enter__`
the yielded value	bound by `as`
code after `yield`	`__exit__`
`try/finally` around `yield`	clean-up despite exceptions

Note

`@contextmanager` is a decorator that turns a generator function into a context manager factory (decorators and generators: Module 2).

More Tools in `contextlib`

Tool	What it does
`suppress(*excs)`	ignore the listed exceptions in the block
`redirect_stdout(f)`	send `print` output to another file object
`chdir(path)`	change the working directory temporarily (3.11+)
`ExitStack()`	manage a *variable* number of context managers
`closing(obj)`	call `obj.close()` at the end of the block
`nullcontext()`	do-nothing stand-in for an optional manager

```python
import io
import os
from contextlib import redirect_stdout, suppress

with suppress(FileNotFoundError):
    os.remove("missing.tmp")

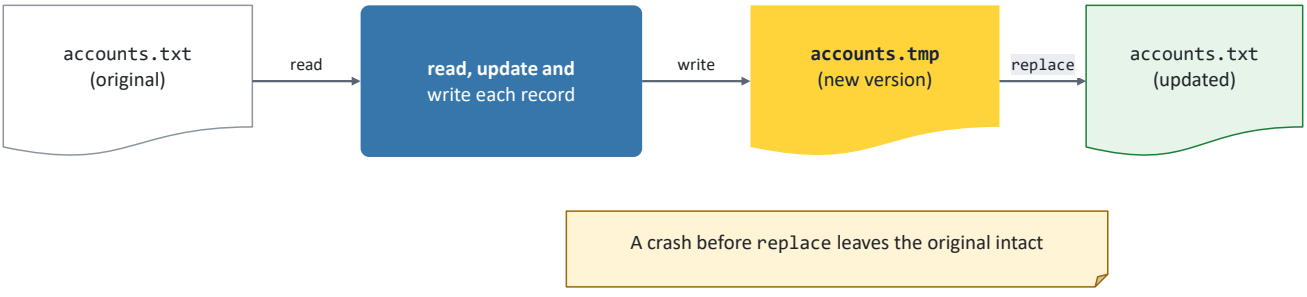
buffer = io.StringIO()
with redirect_stdout(buffer):
    print("captured")
print(buffer.getvalue().upper(), end="")
```

OUTPUT

```
CAPTURED
```


Updating a Text File Safely: The Idea

- Text files cannot be edited *in place*: a longer or shorter record shifts everything after it
- Rewriting the original directly risks a **half-written file** if the program crashes
- Safe pattern: write a complete new version to a temporary file, then **replace** the original



Updating a Text File Safely: The Code

update_accounts.py

```python
from pathlib import Path

source = Path("accounts.txt")
source.write_text("100,Jack,10.0\n300,James,30.0\n", encoding="utf-8")

temp = source.with_suffix(".tmp")
with (
    source.open(encoding="utf-8") as old,
    temp.open("w", encoding="utf-8") as new,
):
    for record in old:
        account, name, balance = record.split(",")
        if account == "300":
            name = "James Sawyer Ford"
        new.write(f"{account},{name},{balance}")
temp.replace(source)  # atomic swap
print(source.read_text(encoding="utf-8"), end="")
```

OUTPUT

```
100,Jack,10.0
300,James Sawyer Ford,30.0
```

Watch out

`Path.replace` (= `os.replace`) is atomic and overwrites the target on every OS. `os.remove` + `os.rename` is not atomic, and `rename` fails on Windows if the target exists.

File-Related Exceptions

exception hierarchy (excerpt)

```
BaseException
 └─ Exception
     ├─ OSError
     │   ├─ FileNotFoundError
     │   ├─ FileExistsError
     │   ├─ PermissionError
     │   ├─ IsADirectoryError
     │   └─ NotADirectoryError
     └─ ValueError
         └─ UnicodeDecodeError
```

```python
def read_config(path: str) -> str:
    """Return the file's text, or "" if it is missing."""
    try:
        with open(path, encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        print(f"{path} not found, using defaults")
        return ""
    except PermissionError as err:
        raise RuntimeError(f"cannot read {path}") from err


print(repr(read_config("settings.ini")))
```

OUTPUT

```
settings.ini not found, using defaults
''
```

FIND THE BUG

Find the Bugs: Appending a Record

The function should **append** one record to the file and return the number of records. Find at least four problems.

```
def add_account(path: str, account: int, name: str,
                balance: float) -> int:
    """Append one record; return the number of records."""
    accounts = open(path, "w")
    accounts.write(f"{account},{name},{balance}")
    return len(accounts.readlines())
```

Intended behaviour:

```
>>> add_account("accounts.txt", 100, "Jack Bauer", 10000.0)
1
>>> add_account("accounts.txt", 200, "Kate Austen", 20000.0)
2
```


ANSWER

Find the Bugs: Appending a Record

- `"w"` empties the file → use `"a"`; no `encoding` → `encoding="utf-8"`
- Record has no `\n`; file never closed → use `with`
- Reading a write-only file raises `io.UnsupportedOperation` → reopen for reading

```
def add_account(path: str, account: int, name: str,
                balance: float) -> int:
    """Append one record; return the number of records."""
    with open(path, "a", encoding="utf-8") as accounts:
        accounts.write(f"{account},{name},{balance}\n")
    with open(path, encoding="utf-8") as accounts:
        return sum(1 for _ in accounts)


add_account("accounts.txt", 100, "Jack Bauer", 10000.0)
print(add_account("accounts.txt", 200, "Kate Austen", 20000.0))
```

OUTPUT

```
2
```

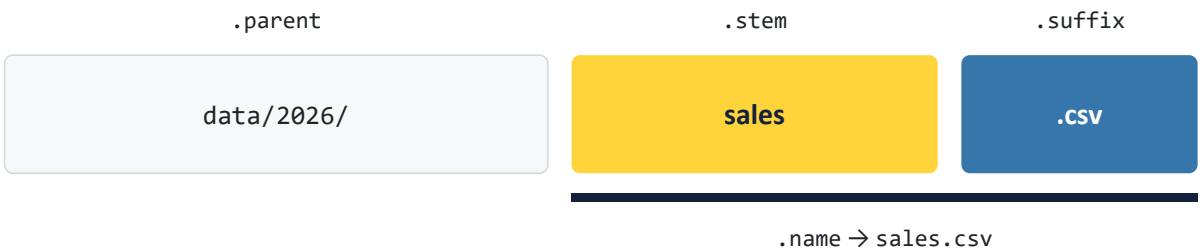
02

Paths and the File System

pathlib, os, shutil and binary data

- Paths as objects with pathlib
- Reading, writing and finding files
- os and shutil for file-system operations
- Binary files and bytes

Paths as Objects With `pathlib`




```python
from pathlib import Path

report = Path("data") / "2026" / "sales.csv"
print(report.as_posix())
print(report.name, report.stem, report.suffix)
print(report.parent.as_posix())
print(report.with_suffix(".json").name)
print(report.is_absolute(), Path.cwd().is_absolute())
```

OUTPUT

```
data/2026/sales.csv
sales.csv sales .csv
data/2026
sales.json
False True
```

Reading and Writing Through `Path`

```python
from pathlib import Path

folder = Path("reports")
folder.mkdir(parents=True, exist_ok=True)
summary = folder / "summary.txt"
summary.write_text("total: 42", encoding="utf-8")

print(summary.exists(), summary.is_file())
print(summary.read_text(encoding="utf-8"))
print(summary.stat().st_size, "bytes")
summary.unlink()
print(summary.exists())
```

OUTPUT

```
True True
total: 42
9 bytes
False
```

`Path` method	Purpose
`read_text()` / `write_text()`	whole text file (pass `encoding=`)
`read_bytes()` / `write_bytes()`	whole binary file
`open(mode, ...)`	file object, for streaming
`exists()`, `is_file()`, `is_dir()`	tests
`mkdir(parents=True, exist_ok=True)`	create folders
`unlink(missing_ok=True)`	delete a file
`stat()`	size, timestamps, permissions

Finding Files: `iterdir`, `glob` and `walk`

```python
from pathlib import Path

root = Path("project")
names = ["a.txt", "b.csv", "sub/c.txt", "sub/deep/d.txt"]
for name in names:
    (root / name).parent.mkdir(parents=True, exist_ok=True)
    (root / name).touch()

print(sorted(p.name for p in root.iterdir()))
print(sorted(p.name for p in root.glob("*.txt")))
print(sorted(p.name for p in root.rglob("*.txt")))
for folder, dirs, files in root.walk():
    print(folder.as_posix(), sorted(files))
```

OUTPUT

```
['a.txt', 'b.csv', 'sub']
['a.txt']
['a.txt', 'c.txt', 'd.txt']
project ['a.txt', 'b.csv']
project/sub ['c.txt']
project/sub/deep ['d.txt']
```

Python version note

`Path.walk()` is new in Python 3.12 — the object-oriented version of `os.walk()`.

From `os.path` to `pathlib`

Task	os / os.path	pathlib
join paths	`os.path.join(d, "a.txt")`	`d / "a.txt"`
name, extension	`os.path.basename(p)`, `splitext(p)`	`p.name`, `p.stem`, `p.suffix`
exists?	`os.path.exists(p)`	`p.exists()`
create folders	`os.makedirs(d, exist_ok=True)`	`d.mkdir(parents=True, exist_ok=True)`
list a folder	`os.listdir(d)`	`d.iterdir()`
delete a file	`os.remove(p)`	`p.unlink()`
rename / replace	`os.rename(a, b)`, `os.replace(a, b)`	`a.rename(b)`, `a.replace(b)`
current folder	`os.getcwd()`	`Path.cwd()`

Tip

Every API that takes a file name also accepts a `Path` (`os.PathLike`): `open(p)`, `shutil.copy(p, q)`, `pd.read_csv(p)`.

The `os` and `shutil` Modules

`os` — operating-system interface

- `os.environ`, `os.getenv("HOME")`
- `os.scandir()`, `os.walk()`
- `os.replace()` — atomic rename
- `os.sep`, `os.linesep`, `os.cpu_count()`

`shutil` — high-level file operations

- `copy()`, `copy2()` (keeps metadata)
- `copytree()`, `move()`
- `rmtree()` — deletes a whole tree!
- `make_archive()`, `disk_usage()`, `which()`

```python
import shutil
from pathlib import Path

photos = Path("photos")
photos.mkdir()
(photos / "cat.jpg").write_bytes(b"\xff\xd8\xff")
shutil.copytree(photos, "backup")
shutil.make_archive("photos", "zip", root_dir="backup")
print([p.name for p in Path("backup").iterdir()])
print(Path("photos.zip").exists())
shutil.rmtree("backup")
```

OUTPUT

```
['cat.jpg']
True
```

Python version note

Python 3.14 adds `Path.copy()`, `copy_into()`, `move()` and `move_into()`.

Binary Files and `bytes`

```
header = bytes([0x89]) + b"PNG\r\n\x1a\n"
with open("image.png", "wb") as f:
    f.write(header)

with open("image.png", "rb") as f:
    data = f.read()
print(data)
print(len(data), data[0], data[1:4])
print(data.startswith(b"\x89PNG"))
```

OUTPUT

```
b'\x89PNG\r\n\x1a\n'
8 137 b'PNG'
True
```

- `"rb"` / `"wb"`: no encoding, no newline translation
- `bytes` is immutable; indexing gives an `int` 0–255
- `bytearray` is the mutable counterpart
- Many formats start with a **magic number**

copy in 1 MiB chunks

```
with (open("big.iso", "rb") as source,
      open("copy.iso", "wb") as target):
    while chunk := source.read(1024 * 1024):
        target.write(chunk)
```


QUIZ

Quick Check: Path Suffixes

What does `Path("backup.tar.gz").suffix` return?

A `".tar.gz"`

B `".gz"`

C `"gz"`

D `"backup.tar"`

ANSWER

Quick Check: Path Suffixes

What does `Path("backup.tar.gz").suffix` return?

A `".tar.gz"`

B `".gz"`

C `"gz"`

D `"backup.tar"`

Why

`suffix` is the **last** extension, including the dot. `suffixes` returns `['.tar', '.gz']` and `stem` is `"backup.tar"`.

TRY IT

Find the Largest Files

Write `largest_files(folder, n)` that returns the *n* largest files anywhere below `folder` as `(relative path, size in bytes)` tuples, largest first.

sample folder

```
demo/
├─ a.txt        10 bytes
├─ c.log        50 bytes
└─ img/
   └─ b.png    300 bytes
```

```
>>> largest_files(Path("demo"), 2)
[('img/b.png', 300), ('c.log', 50)]
```

Tip

Hints: `rglob("*")`, `is_file()`, `stat().st_size`, `relative_to(folder)`, `sorted(..., key=..., reverse=True)`.

ANSWER

Find the Largest Files

```python
from pathlib import Path


def largest_files(folder: Path, n: int) -> list[tuple[str, int]]:
    """Return (relative path, size) of the n largest files."""
    sizes = [(p.relative_to(folder).as_posix(), p.stat().st_size)
             for p in folder.rglob("*") if p.is_file()]
    return sorted(sizes, key=lambda item: item[1], reverse=True)[:n]


root = Path("demo")
(root / "img").mkdir(parents=True)
(root / "a.txt").write_bytes(b"x" * 10)
(root / "img" / "b.png").write_bytes(b"x" * 300)
(root / "c.log").write_bytes(b"x" * 50)
print(largest_files(root, 2))
```

OUTPUT

```
[('img/b.png', 300), ('c.log', 50)]
```


03

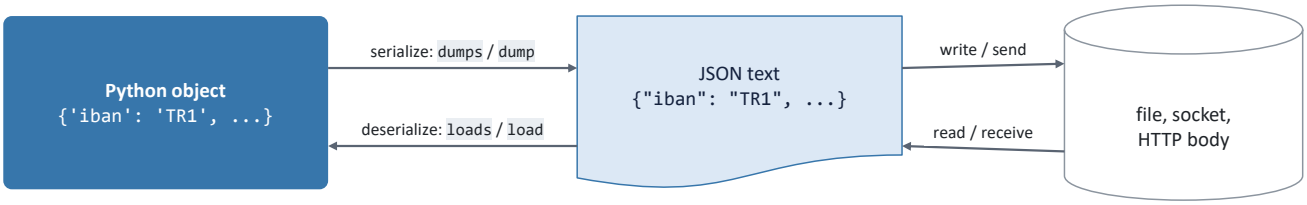
Serializing Data With JSON

Turning Python objects into portable, human-readable text — and back

- Serialization and the JSON data format
- dump, load, dumps and loads
- Invalid JSON and command-line tools
- Custom types: dataclasses and dates

Serialization and Deserialization

- **Serialization**: converting an in-memory object into text or bytes, to store or transmit it
- **Deserialization**: rebuilding an equivalent object from that representation
- **JSON** (JavaScript Object Notation): text-based, language-independent, human-readable — the standard format of web APIs



The JSON Data Format

account.json

```
{
  "iban": "TR1",
  "fullname": "Jack Bauer",
  "balance": 10000.0,
  "active": true,
  "cards": ["debit", "credit"],
  "manager": null
}
```

JSON	Python
object `{...}`	`dict`
array `[...]`	`list` (`tuple` when writing)
string `"..."`	`str`
number (integer / real)	`int` / `float`
`true` / `false`	`True` / `False`
`null`	`None`

Watch out

Double quotes only, keys are always strings, no comments, no trailing commas.

Writing JSON With `json.dump`

save_accounts.py

```python
import json

accounts = [
    {"iban": "TR1", "owner": "Kate Austen", "balance": 100000.0},
    {"iban": "TR2", "owner": "Ben Linus", "balance": 300000.0},
]

with open("accounts.json", "w", encoding="utf-8") as json_file:
    json.dump(accounts, json_file, indent=2)

with open("accounts.json", encoding="utf-8") as json_file:
    print(json_file.read())
```

OUTPUT

```
[
  {
    "iban": "TR1",
    "owner": "Kate Austen",
    "balance": 100000.0
  },
  {
    "iban": "TR2",
    "owner": "Ben Linus",
    "balance": 300000.0
  }
]
```

Key idea

`json.dump(obj, file)` serializes *obj* and writes the text to an open file; `indent=2` pretty-prints it.

Reading JSON With `json.load`

load_accounts.py

```python
import json

with open("accounts.json", encoding="utf-8") as json_file:
    accounts = json.load(json_file)

print(type(accounts).__name__, len(accounts))
for account in accounts:
    print(account["owner"], account["balance"] + 500)
print(type(accounts[0]["balance"]).__name__)
```

OUTPUT

```
list 2
Kate Austen 100500.0
Ben Linus 300500.0
float
```

Key idea

Reads `accounts.json` from the previous slide: a `list` of two `dict` objects.

Note

`json.load` returns plain `dict`, `list`, `str`, number, `bool` and `None` objects — never instances of your own classes.

JSON Strings: `dumps` and `loads`

```
>>> import json
>>> account = {"owner": "Işık", "tags": ("vip",)}
>>> json.dumps(account)
'{"owner": "I\\u015f\\u0131k", "tags": ["vip"]}'
>>> json.dumps(account, ensure_ascii=False)
'{"owner": "Işık", "tags": ["vip"]}'
>>> print(json.dumps(account, indent=2))
{
  "owner": "Işık",
  "tags": [
    "vip"
  ]
}
>>> json.loads('{"balance": 100.5, "active": false}')
{'balance': 100.5, 'active': False}
```

Function	Converts
`dump(obj, file)`	object → JSON in a file
`dumps(obj)`	object → JSON `str`
`load(file)`	JSON in a file → object
`loads(s)`	JSON `str` or `bytes` → object

Tip

The **s** stands for *string*. `ensure_ascii=False` keeps non-ASCII characters readable.

Invalid JSON and Command-Line Tools

```
import json

bad = '{"iban": "TR1", "balance": 100.0,}'
try:
    json.loads(bad)
except json.JSONDecodeError as err:
    print(err.msg)
    print(f"line {err.lineno}, column {err.colno}")
```

OUTPUT

```
Illegal trailing comma before end of object
line 1, column 33
```

- `JSONDecodeError` is a subclass of `ValueError`
- Typical causes: single quotes, trailing commas, comments, `NaN`

TERMINAL

```
$ python -m json.tool accounts.json
[
    {
        "iban": "TR1",
        "owner": "Kate Austen"
    }
]
$ python -m json accounts.json   # 3.14+
```

Tip

`json.tool` validates and pretty-prints JSON files without writing any code.

Encoding Custom Types With `default`

```python
import json
from dataclasses import asdict, dataclass
from datetime import date


@dataclass
class Account:
    iban: str
    opened: date


def encode_extra(obj: object) -> str:
    """Serialize types that json cannot handle."""
    if isinstance(obj, date):
        return obj.isoformat()
    raise TypeError(f"cannot serialize {type(obj).__name__}")


account = Account("TR1", date(2026, 9, 1))
print(json.dumps(asdict(account), default=encode_extra))
```

OUTPUT

```
{"iban": "TR1", "opened": "2026-09-01"}
```

- `asdict` turns a dataclass into a `dict`
- `json` calls `default(obj)` for every object it cannot serialize
- Return something serializable — or raise `TypeError`
- Dates become ISO 8601 strings

Decoding Custom Types With `object_hook`

```python
import json
from dataclasses import dataclass
from datetime import date


@dataclass
class Account:
    iban: str
    opened: date


def decode_account(obj: dict) -> object:
    """Build an Account from dicts with exactly its keys."""
    if obj.keys() == {"iban", "opened"}:
        opened = date.fromisoformat(obj["opened"])
        return Account(obj["iban"], opened)
    return obj


text = '[{"iban": "TR1", "opened": "2026-09-01"}]'
print(json.loads(text, object_hook=decode_account))
```

OUTPUT

```
[Account(iban='TR1',
         opened=datetime.date(2026, 9, 1))]
```

- `object_hook` is called for **every** decoded JSON object, innermost first
- Return a replacement, or the `dict` unchanged
- JSON has no type tags — you decide how to recognise your objects

TRACE THE CODE

A JSON Round Trip

```python
import json

data = {1: (1, 2), "ok": True, "note": None, "ratio": 0.5}
text = json.dumps(data)
print(text)
restored = json.loads(text)
print(restored)
print(restored == data)
```

Tip

Predict all three lines before running the code. Look closely at the key `1` and the tuple `(1, 2)`.

ANSWER

A JSON Round Trip

OUTPUT

```
{"1": [1, 2], "ok": true, "note": null, "ratio": 0.5}
{'1': [1, 2], 'ok': True, 'note': None, 'ratio': 0.5}
False
```

- JSON keys are always strings: the key `1` becomes `"1"`
- The tuple `(1, 2)` becomes an array, and comes back as a **list**
- `True`/`None` → `true`/`null` → `True`/`None` survive unchanged
- So `restored != data`: a JSON round trip is not always lossless

TRY IT

Load Settings With Defaults

Write `load_settings(path, defaults)` that returns a **new** dict: the defaults updated with the values stored in the JSON file *path*.

- A missing file means "use the defaults"
- Invalid JSON raises `ValueError` mentioning the file name

```
>>> load_settings("settings.json", {"theme": "light", "size": 12})
{'theme': 'dark', 'size': 12}
```

Tip

Hints: `Path.read_text`, `json.loads`, `except FileNotFoundError`, `raise ... from`, and `defaults | stored` for the merge.

ANSWER

Load Settings With Defaults

```python
import json
from pathlib import Path


def load_settings(path: str, defaults: dict) -> dict:
    """Return defaults updated with the settings in path."""
    try:
        text = Path(path).read_text(encoding="utf-8")
    except FileNotFoundError:
        return dict(defaults)
    try:
        stored = json.loads(text)
    except json.JSONDecodeError as err:
        raise ValueError(f"invalid settings in {path}") from err
    return defaults | stored


Path("settings.json").write_text('{"theme": "dark"}', encoding="utf-8")
print(load_settings("settings.json", {"theme": "light", "size": 12}))
print(load_settings("missing.json", {"theme": "light"}))
```

OUTPUT

```
{'theme': 'dark', 'size': 12}
{'theme': 'light'}
```

Key idea

`defaults | stored` builds a **new** dict; values from the file win.

Note

`raise ... from err` keeps the original `JSONDecodeError` as `__cause__`.

04

CSV Files and Other Formats

Tabular data with the csv module, a word on pickle, and choosing a format

- csv.reader, csv.writer and newline=""
- DictReader, DictWriter and dialects
- CSV with pandas
- pickle (and why it is dangerous), binary formats

The CSV Format

- **CSV** (comma-separated values): one record per line, fields split by a delimiter
- Supported by every spreadsheet and database; compact for large tables
- Loosely standardised (RFC 4180): fields containing commas, quotes or newlines are **quoted**, inner quotes doubled
- Everything is text: no types, no nesting

world.csv

```
code,name,continent,population
tur,Türkiye,asia,85000000
usa,"United States, The",america,335000000
xyz,"The ""Quoted"" Land",europe,1000
```

Watch out

Never parse CSV with `line.split(",")` — quoted fields break it. Use the `csv` module.

Writing Rows With `csv.writer`

write_world.py

```python
import csv

countries = [
    ("tur", "Türkiye", "asia", 85_000_000),
    ("fra", "France", "europe", 68_000_000),
    ("usa", "United States, The", "america", 335_000_000),
]
with open("world.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(("code", "name", "continent", "population"))
    writer.writerows(countries)

with open("world.csv", encoding="utf-8") as f:
    print(f.read(), end="")
```

OUTPUT

```
code,name,continent,population
tur,Türkiye,asia,85000000
fra,France,europe,68000000
usa,"United States, The",america,335000000
```

- `writerow` writes one sequence, `writerows` many
- Values are converted with `str()`; quoting is automatic

Reading Rows With `csv.reader`

read_world.py

```python
import csv

with open("world.csv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)
    for code, name, continent, population in reader:
        print(f"{name:<20}{continent:<9}{int(population):>13,}")
print(header)
```

OUTPUT

```
Türkiye             asia        85,000,000
France              europe      68,000,000
United States, The  america    335,000,000
['code', 'name', 'continent', 'population']
```

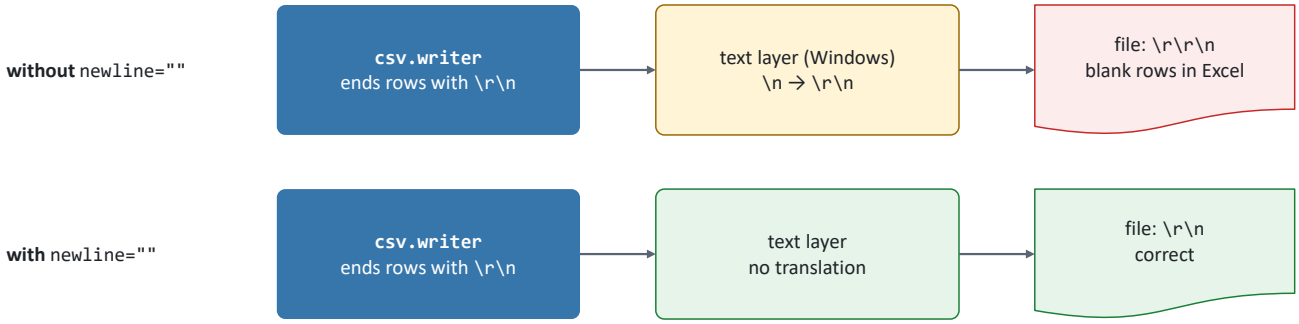
Key idea

Each row is a `list` of `str` — convert types yourself: `int(population)`.

Tip

`next(reader)` consumes the header row; quoted commas stay inside their field.

Why `newline=""` Matters




```
>>> import csv
>>> import io
>>> buffer = io.StringIO()
>>> csv.writer(buffer).writerow(["Ada", 'say "hi"', "a,b"])
24
>>> buffer.getvalue()
'Ada,"say ""hi""","a,b"\r\n'
```

Key idea

The `csv` module handles line endings and quoted line breaks itself. Always open CSV files with `newline=""`.

Writing Records With `DictWriter`

dict_writer.py

```python
import csv

countries = [
    {"code": "tur", "name": "Türkiye", "population": 85_000_000},
    {"code": "jpn", "name": "Japan", "population": 124_000_000},
]

with open("world.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["code", "name", "population"])
    writer.writeheader()
    writer.writerows(countries)

with open("world.csv", encoding="utf-8") as f:
    print(f.read(), end="")
```

OUTPUT

```
code,name,population
tur,Türkiye,85000000
jpn,Japan,124000000
```

- `fieldnames` fixes the column order; `writeheader()` writes it as the first row
- Missing keys are written as `restval` (default `""`); unknown keys raise `ValueError` unless `extrasaction="ignore"`

Reading Records With `DictReader`

dict_reader.py

```python
import csv
from collections import Counter

totals: Counter[str] = Counter()
with open("world.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        totals[row["continent"]] += int(row["population"])
print(row)
print(dict(totals))
```

world.csv

```
code,name,continent,population
fra,France,europe,68000000
tur,Türkiye,asia,85000000
ita,Italy,europe,59000000
```

Key idea

`DictReader` uses the first row as keys. Every value is still a `str`.

OUTPUT

```
{'code': 'ita', 'name': 'Italy', 'continent': 'europe', 'population': '59000000'}
{'europe': 127000000, 'asia': 85000000}
```


Dialects and Formatting Options

Parameter	Default	Other values
`delimiter`	`","`	`";"` (Excel, Turkish locale), `"\t"` (TSV)
`quotechar`	`'"'`	any single character
`quoting`	`QUOTE_MINIMAL`	`QUOTE_ALL`, `QUOTE_NONNUMERIC`, `QUOTE_STRINGS` (3.12+)
`lineterminator`	`"\r\n"`	`"\n"` (writer only)
`dialect`	`"excel"`	`"excel-tab"`, `"unix"`, or `csv.register_dialect(...)`

```python
import csv
import io

buffer = io.StringIO()
writer = csv.writer(buffer, delimiter=";", quoting=csv.QUOTE_NONNUMERIC)
writer.writerow(["Türkiye", 85.5, "asia"])
print(repr(buffer.getvalue()))
print(csv.Sniffer().sniff("code;name\ntur;Türkiye\n").delimiter)
```

OUTPUT

```
'"Türkiye";85.5;"asia"\r\n'
;
```

CSV With pandas

pandas_csv.py

```python
import pandas as pd

countries = [
    ("tur", "Türkiye", "asia", 85_000_000),
    ("fra", "France", "europe", 68_000_000),
    ("ita", "Italy", "europe", 59_000_000),
]
columns = ["code", "name", "continent", "population"]
df = pd.DataFrame(countries, columns=columns)
df.to_csv("world_pandas.csv", index=False)

df = pd.read_csv("world_pandas.csv")
print(df.dtypes["population"])
print(df.groupby("continent")["population"].sum())
```

TERMINAL

```
$ pip install pandas
```

OUTPUT

```
int64
continent
asia       85000000
europe    127000000
Name: population, dtype: int64
```

Key idea

`read_csv` infers column types; `index=False` stops `to_csv` from writing the row index as an extra column.

Binary Serialization With `pickle`

```python
import pickle
from dataclasses import dataclass
from datetime import date


@dataclass
class Account:
    iban: str
    opened: date


accounts = [Account("TR1", date(2026, 9, 1))]
with open("accounts.pkl", "wb") as f:
    pickle.dump(accounts, f)
with open("accounts.pkl", "rb") as f:
    restored = pickle.load(f)
print(restored == accounts, type(restored[0]).__name__)
```

OUTPUT

```
True Account
```

Watch out

Never unpickle data from an untrusted source: `pickle.load` can execute **arbitrary code**.

- Serializes almost any Python object, incl. custom classes
- Binary and Python-only — not for data exchange
- Tied to your class definitions: renaming a class breaks old files
- Use for private caches and Python-to-Python transfer

Choosing a Data Format

Format	Readable	Data model	Safe for untrusted input	Typical use
JSON	yes	nested objects, arrays, basic types	yes	web APIs, configuration, documents
CSV	yes	flat table of strings	yes	spreadsheets, bulk import/export
XML	yes	tree with attributes; schemas	with `defusedxml`	documents, enterprise, office formats (next)
`pickle`	no	almost any Python object	**no**	private caches, Python IPC
Protobuf, Avro, Parquet	no	typed schema, binary	yes	high-volume services, analytics

Key idea

Choose the simplest format your consumers can read: JSON for structured data, CSV for tables, a schema-based binary format when size and speed matter.

QUIZ

Quick Check: `DictReader` Values

`world.csv` contains the row `tur,Türkiye,85000000`. It is read with `csv.DictReader`. What is `type(row["population"])`?

A `int`

B `str`

C `float`

D It depends on the value

ANSWER

Quick Check: `DictReader` Values

`world.csv` contains the row `tur,Türkiye,85000000`. It is read with `csv.DictReader`. What is `type(row["population"])`?

- A `int`
- B `str` ✓
- C `float`
- D It depends on the value

Why

The `csv` module never converts types: every field is a `str`. Convert explicitly with `int(...)` or `float(...)` — or use pandas, which infers column types.

TRY IT

Convert CSV to JSON

Read `world.csv` (columns `code,name,continent,population`) and write `world.json`: an object that maps each continent to the list of its country names, most populous first.

world.csv

```
code,name,continent,population
fra,France,europe,68000000
tur,Türkiye,asia,85000000
ita,Italy,europe,59000000
```

world.json (expected)

```
{"asia": ["Türkiye"],
 "europe": ["France", "Italy"]}
```

Tip

Hints: `csv.DictReader`, `sorted(..., key=...)`, `dict.setdefault`, `json.dumps(..., ensure_ascii=False)`.

ANSWER

Convert CSV to JSON

csv_to_json.py

```python
import csv
import json
from pathlib import Path

with open("world.csv", newline="", encoding="utf-8") as f:
    rows = sorted(csv.DictReader(f), key=lambda r: -int(r["population"]))

groups: dict[str, list[str]] = {}
for row in rows:
    groups.setdefault(row["continent"], []).append(row["name"])

text = json.dumps(groups, ensure_ascii=False)
Path("world.json").write_text(text, encoding="utf-8")
print(text)
```

OUTPUT

```
{"asia": ["Türkiye"], "europe": ["France", "Italy"]}
```

Key idea

Sorting first means each continent's list is already in population order when `setdefault` appends to it.

05

XML Fundamentals

Structure, syntax rules and namespaces of the Extensible Markup Language

- Documents, elements and attributes
- Entities, CDATA, comments and white space
- Well-formedness rules
- Namespaces; XML versus HTML

What Is XML?

- **XML** (Extensible Markup Language): a text format for *structured data*
- You invent the tags: the vocabulary is **extensible**
- Self-describing: the markup names the data it wraps
- W3C Recommendation (1998), a simplified subset of SGML
- Read and written by **XML processors** (parsers)

Office documents
`.docx`, `.xlsx`, `.pptx` are ZIP archives of XML files

Graphics and feeds
SVG images, RSS and Atom news feeds

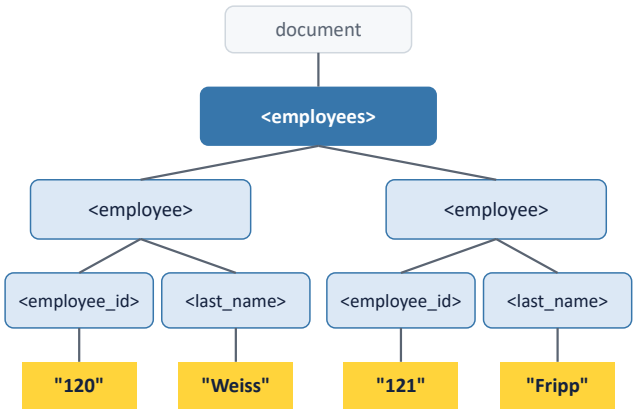
Configuration
Maven `pom.xml`, Android layouts, `.csproj`

Data exchange
SOAP web services, UBL e-invoices (e-Fatura)

XML Documents Are Trees

employees.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<employees>
  <employee>
    <employee_id>120</employee_id>
    <last_name>Weiss</last_name>
  </employee>
  <employee>
    <employee_id>121</employee_id>
    <last_name>Fripp</last_name>
  </employee>
</employees>
```



Key idea

Exactly one **root element**; every other element has exactly one parent. Blue boxes are element nodes, yellow boxes are text nodes.

Anatomy of an XML Document

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <book>
3    <title>Alphabet from A to Z</title>
4    <isbn number="1112-23-4356"/>
5    <author>
6      <firstName>Boreng</firstName>
7      <lastName>Riter</lastName>
8    </author>
9    <chapter title="Letter A">
10     The letter A is the first letter.
11   </chapter>
12   <!-- the other chapters are missing -->
13   <chapter title="Letter Z">
14     The letter Z is the last letter.
15   </chapter>
16 </book>
```

Lines	What it is
1	XML declaration (optional, must come first)
2, 16	Root element: start and end tag
3	Child element with character data
4	Empty element with an attribute
5–8	Element containing nested child elements
9–11	Attribute plus parsed character data
12	Comment (not part of the data)

Prolog, Root Element and Epilog

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!-- prolog: declaration, comments, DOCTYPE, PIs -->
<?xml-stylesheet type="text/xsl" href="staff.xsl"?>
<employees>
  <employee>...</employee>
</employees>
<!-- epilog: only comments and PIs may follow -->
```

Prolog — XML declaration, comments, DOCTYPE, processing instructions

Root element — the one element that contains all data

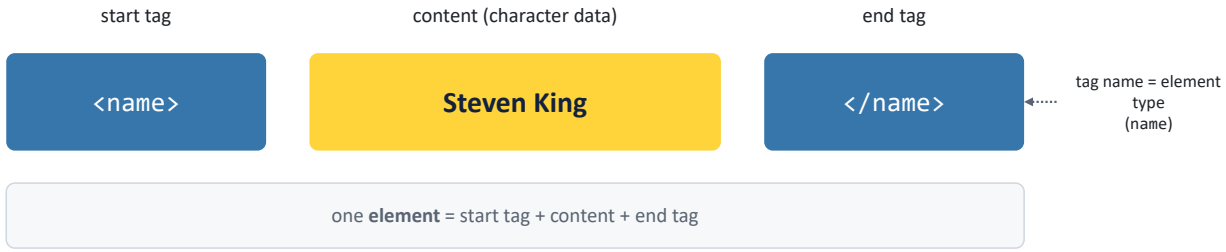
Epilog — optional comments and processing instructions

Declaration attribute	Rule
`version`	Required: `"1.0"` (1.1 is rarely used)
`encoding`	Optional; default UTF-8 (or UTF-16 with a BOM)
`standalone`	Optional; `"yes"` = no external markup declarations needed

Watch out

If present, the declaration must be the **very first** characters of the file — not even a blank line or comment before it.

Elements: Tags and Content




```xml
<employee>
  <name>Steven King</name>
  <initials/>        <!-- empty element -->
  <phone></phone>    <!-- also empty -->
</employee>
```

- Content: text, child elements, or both
- Empty element: `<initials/>` ≡ `<initials></initials>`
- Empty elements may still carry attributes

Nesting Elements Correctly

Wrong — the tags overlap

```xml
<employee>
  <first>Steven<last>King</first></last>
</employee>
```

Right — each element closes inside its parent

```xml
<employee>
  <first>Steven</first><last>King</last>
</employee>
```

- Exactly **one root element** per document
- Every start tag has a matching end tag (or is self-closing)
- Close tags in reverse order of opening, like a stack
- Names are case-sensitive: `<Employee>` ≠ `<employee>`

> ✓ **Tip**
> A parser can check nesting with a simple stack: push on a start tag, pop and compare on an end tag.

XML Names

- Letters (any script), digits, `_`, `-`, `.`
- Must start with a letter or `_` — not a digit, `-` or `.`
- No spaces, no `@ # * ; $ %`
- `:` only between a namespace prefix and a local name
- Names starting with `xml` (any case) are reserved
- Case-sensitive; no length limit

Name	Legal?
`<Drivers_License_Number>`	yes
`<month-day-year>`	yes
`<_4-lane>`, `<_12/>`	yes
`<téléphone>`	yes (non-English letters)
`<xsl:copy-of>`	yes (prefix `xsl`)
`<1ex/>`	no: starts with a digit
`<last@name>`, `<A*2/>`	no: `@` and `*` not allowed
`<first name>`	no: space ends the name

Attributes Versus Child Elements

- **Attribute**: a `name="value"` pair inside the start tag
- Value always quoted: `"..."` or `'...'`
- Names unique within one element; case-sensitive
- A single string value: no nesting, no order

> ✓ **Tip**
> Rule of thumb: the *data* goes into elements; *metadata* (ids, units, language, flags) goes into attributes.

As child elements

```xml
<employee>
  <id>100</id>
  <last_name>King</last_name>
  <salary>24000</salary>
</employee>
```

As attributes

```xml
<employee id="100" last_name="King"
          salary="24000"/>
```

Entity and Character References

Reference	Character	When required
`<`	`<`	always in text and attribute values
`&`	`&`	always in text and attribute values
`>`	`>`	only in the sequence `]]>`
`"`	`"`	inside a `"`-quoted attribute
`'`	`'`	inside a `'`-quoted attribute

- Character reference: `©` (decimal) or `©` (hex) → ©
- Every reference ends with a semicolon `;`

```
<publisher>O'Reilly &amp; Associates</publisher>
<rule>salary &lt; 1000</rule>
<name nick='Rachael O&apos;Leary'/>
<note>&#169; 2026 &#x2014; all rights</note>
```

Key idea

The parser replaces each reference, so the application receives `O'Reilly & Associates` and `salary < 1000`.

CDATA Sections

```
<script><![CDATA[
function inRange(a, b) {
  return a < b && b < 100;
}
]]></script>
<snippet><![CDATA[<dog breed="spaniel"/>]]></snippet>
```

- Syntax: `<![CDATA[ … ]]>`
- Content is taken literally: no tags, no references
- Only forbidden text inside: `]]>` itself
- Ideal for embedded code or markup shown as text

```
>>> import xml.etree.ElementTree as ET
>>> ET.fromstring("<s><![CDATA[a < b && c]]></s>").text
'a < b && c'
>>> ET.fromstring("<s><![CDATA[&#38;]]></s>").text
'&#38;'
```

Common misconception

Inside CDATA, `&` or `&` are **not** decoded — they stay exactly as typed.

Comments and White Space

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- Employees of the company,
     exported from the HR system -->
<employees>
  <name>Steven King</name>  <!-- full name -->
</employees>
```

- Comments: `<!--` … `-->`, may span lines
- Anywhere except inside a tag or another comment
- `--` may not appear inside a comment
- Not data: parsers may drop them

```
>>> import xml.etree.ElementTree as ET
>>> ET.fromstring("<name>\r\n  Ada \r\n</name>").text
'\n  Ada \n'
```

- White space = space, tab, line feed, carriage return
- All white space in content reaches the program
- Line ends are normalised to `\n` (line feed)
- Indentation becomes text too (see DOM, Section 8)

Well-Formed XML: The Rules

1. One root
Exactly one top-level element

2. Matching tags
Every start tag closed, or `<empty/>`

3. Proper nesting
No overlapping elements

4. Case matters
`<a>` must end with `</a>`, not `</A>`

5. Quoted values
Attribute values always in quotes

6. Unique attributes
No attribute twice in one tag

7. Escape < and &
Use `<` and `&` in text and values

8. Declaration first
Nothing before `<?xml ...?>`

Key idea

A document that breaks any rule is **not XML**. A conforming parser must stop with a fatal error — no guessing, no repair (unlike HTML browsers).

Checking Well-Formedness

check_xml.py

```python
import xml.etree.ElementTree as ET

SNIPPETS = [
    "<question>Is this legal?</question>",
    "<question>Is this legal?</Question>",
    "<rule>salary < 1000</rule>",
]

for text in SNIPPETS:
    try:
        ET.fromstring(text)
        print("OK   ", text)
    except ET.ParseError as err:
        print("ERROR", err)
```

OUTPUT

```
OK    <question>Is this legal?</question>
ERROR mismatched tag: line 1, column 26
ERROR not well-formed (invalid token): line 1, column 14
```

- Editors check as you type: VS Code (XML extension), PyCharm
- `xmllint` (libxml2) on the command line:

TERMINAL

```
$ xmllint --noout employees.xml
$ echo $?
0
```

Note

`ParseError` carries the line and column of the first error — the parser stops there.

FIND THE BUG

Class Activity: Spot the Errors

Each line is meant to be a complete XML document. What is wrong with each one?

```
1  <?xml version="1.0"?><question>Is this legal?</question><answer>No</answer>
2  <!-- An XML document --><?xml version="1.0"?><root/>
3  <question 4you="Is this attribute name correct"/>
4  <email id=mark.ant@example.com></email>
5  <question>Is this legal</Question>
```

ANSWER

Class Activity: Spot the Errors

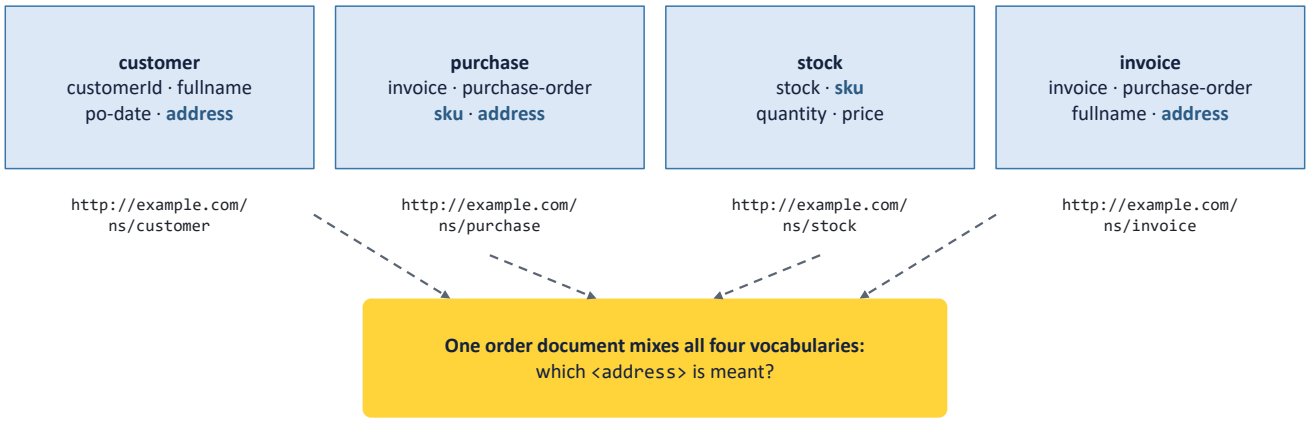
#	Error	Parser message
1	Two top-level elements: only one root allowed	junk after document element
2	Comment before the XML declaration	XML declaration not at start
3	Attribute name starts with a digit	not well-formed (invalid token)
4	Attribute value not quoted	not well-formed (invalid token)
5	`question` vs `Question`: names are case-sensitive	mismatched tag

Tip

Fixes: wrap 1 in one root; delete the comment or move it after the declaration; rename to `you4`; quote the value; use the same case.

5 · XML FUNDAMENTALS

Why Namespaces?



- Different vocabularies reuse the same names (`address`, `sku`, `invoice`)
- A **namespace** qualifies each name with a globally unique URI
- Makes mixed documents unambiguous: XHTML + SVG, SOAP envelopes, XSLT

Declaring and Using Namespaces

```
<order xmlns="http://example.com/ns/purchase"
       xmlns:c="http://example.com/ns/customer">
  <sku>A-1001</sku>
  <address>Istanbul warehouse</address>
  <c:fullname>Ada Lovelace</c:fullname>
  <c:address>London</c:address>
</order>
```

Syntax	Meaning
`xmlns="URI"`	Default namespace for unprefixed element names
`xmlns:c="URI"`	Binds the prefix `c` to a URI
`c:address`	Qualified name: prefix + local name
Scope	The declaring element and all its descendants
Attributes	Unprefixed attributes are in *no* namespace

Common misconception

The URI is only an **identifier** — the parser never downloads it.

XML Versus HTML

Aspect	XML	HTML
Purpose	Describe and exchange **data**	Present documents in a **browser**
Tags	User-defined vocabularies	Fixed set defined by the HTML standard
Errors	Fatal: parser must stop	Tolerated: browsers repair the markup
Case	Case-sensitive names	Case-insensitive tags (`<P>` = `<p>`)
End tags	Always required (or `<x/>`)	Often optional (`<p>`, `<li>`)
White space	Preserved and passed on	Collapsed when rendered

Note

XHTML is HTML written as well-formed XML. Today's HTML Living Standard (WHATWG) defines an HTML syntax and an XML syntax for the same document model.

FIND THE BUG

Find the Errors: Attribute Syntax

```
1  <root>
2    <e1 a*b = "23432"/>
3    <e2 value = "12'/>
4    <e3 value="aa"aa"/>
5    <e4 value=bbbb/>
6    <e5 xml-ID = "xml2"/>
7  </root>
```

Which lines are not well-formed? Is anything legal but still a bad idea?

ANSWER

Find the Errors: Attribute Syntax

Line	Verdict
2	Error: `*` is not allowed in a name
3	Error: opened with `"`, closed with `'`
4	Error: `"` inside a `"`-quoted value — use `'aa"aa'` or `"`
5	Error: value not quoted
6	Well-formed, but names starting with `xml` are reserved — avoid

✗ **Common misconception**

Spaces around `=` are allowed: `value = "12"` is fine.

FIND THE BUG

Find the Errors: Text Content

Snippet A

```
1  <root>
2    <example>
3      <![CDATA[ <P>Q&R]]>
4    </example>
5    <Name>
6      Binnur Kurt
7    </Name>
8    <Address/>
9  </root>
```

Snippet B

```
1  <root>
2    <isLower>
3      23 < 46
4    </isLower>
5    <Name>
6      Wiley & Sons
7    </name>
8  </root>
```


ANSWER

Find the Errors: Text Content

Key idea

A is well-formed. CDATA protects `<P>` and `&`; indentation is just text; `<Address/>` is an empty element.

- B line 3: raw `<` → `23 < 46`
- B line 6: raw `&` → `Wiley & Sons`
- B lines 5/7: `<Name>` … `</name>` differ in case

```
<isLower>23 &lt; 46</isLower>
<Name>Wiley &amp; Sons</Name>
```

QUIZ

Quick Check: Well-Formed or Not?

Which of these elements is well-formed?

A `<price currency=EUR>9.99</price>`

B `<note>5 < 7</note>`

C `<note>5 < 7 & 8 > 2</note>`

D `<Note>ok</note>`

ANSWER

Quick Check: Well-Formed or Not?

Which of these elements is well-formed?

A `<price currency=EUR>9.99</price>`

B `<note>5 < 7</note>`

C `<note>5 < 7 & 8 > 2</note>`

D `<Note>ok</note>`

★ **Why**

Only `<` and `&` must be escaped in text; a literal `>` is allowed. A needs quotes, B has a raw `<`, D mixes `Note` and `note`.

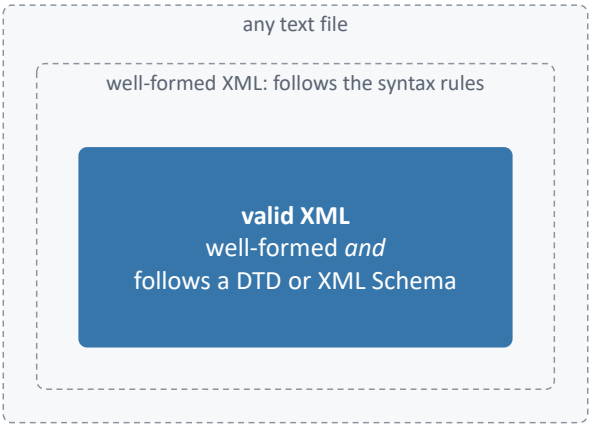
06

Validating XML with a DTD

Document Type Definitions: grammar rules for elements, attributes and entities

- Well-formed versus valid
- Element declarations and cardinality
- Attribute declarations and defaults
- Entities; validating with xmllint

Well-Formed Versus Valid



- **Well-formed**: syntax only — any parser checks it
- **Valid**: also matches a grammar (a *schema*)
- The grammar encodes business rules: required children, order, allowed values
- Schema languages: **DTD**, **XML Schema (XSD)**, RELAX NG
- Validate at the boundary, then trust the data inside

What Is a DTD?

- A **document type definition** (DTD) is the grammar of a class of documents
- Declares elements, attributes, entities and notations
- Stored in the document (*internal subset*) or in a `.dtd` file (*external*)
- Case-sensitive; comments allowed, but not inside a declaration
- Anything that is not declared is **forbidden**

Declares	Syntax
Element	`<!ELEMENT element-name content-model>`
Attribute	`<!ATTLIST element-name attribute-name type default>`
Entity	`<!ENTITY entity-name "replacement text">`
Notation	`<!NOTATION notation-name SYSTEM "identifier">`

A First DTD

employees.dtd

```
<!ELEMENT employees (employee)>
<!ELEMENT employee (name)>
<!ELEMENT name (#PCDATA)>
```

- `(employee)`: exactly one `employee` child
- `(#PCDATA)`: text only, no child elements
- Every element used must be declared

Valid

```
<employees>
  <employee>
    <name>Steven King</name>
  </employee>
</employees>
```

Invalid: two `employee` children

```
<employees>
  <employee><name>Steven King</name></employee>
  <employee><name>Ellen Abel</name></employee>
</employees>
```

Linking a Document to Its DTD

internal subset

```
<?xml version="1.0"?>
<!DOCTYPE employees [
  <!ELEMENT employees (#PCDATA)>
]>
<employees>Employee data</employees>
```

> **Watch out**
> The name after `DOCTYPE` must be the **root element** name.

Form	Document type declaration
Internal	`<!DOCTYPE employees [ ... ]>`
External file	`<!DOCTYPE employees SYSTEM "emp.dtd">`
Public	`<!DOCTYPE html PUBLIC "fpi" "url.dtd">`
Both	`<!DOCTYPE employees SYSTEM "emp.dtd" [...]>`

- Placed after the XML declaration, before the root
- Internal declarations are read first and win for entities
- `PUBLIC` needs a public id **and** a system URL

Element Content Models

Content model	Declaration	Allows
Empty	`<!ELEMENT job EMPTY>`	No content (attributes allowed)
Sequence	`<!ELEMENT employee (employee_id, last_name, job_id)>`	These children, in order
Choice	`<!ELEMENT job_id (manager | worker)>`	Exactly one of the alternatives
Nested groups	`<!ELEMENT hire_date (date | (day, month, year))>`	Groups combined freely
Text	`<!ELEMENT last_name (#PCDATA)>`	Character data only
Mixed	`<!ELEMENT note (#PCDATA | b | i)*>`	Text interleaved with listed elements
Any	`<!ELEMENT extra ANY>`	Any declared elements and text

> **Watch out**
> Mixed content has one fixed shape: `(#PCDATA | a | b)*` — `#PCDATA` first, `|` only, and the `*` is required.

Cardinality: ?, * and +

Suffix	Occurrences
(none)	exactly one
`?`	zero or one (optional)
`*`	zero or more
`+`	one or more

Key idea

A suffix applies to the element or the parenthesised group just before it.

cardinality examples

```
<!ELEMENT employees  (employee*)>
<!ELEMENT department (employee+)>
<!ELEMENT employee
    (last_name, salary, commission_pct?)>
<!ELEMENT jobs  (president, (manager | clerk)*)>
<!ELEMENT phones (phone)+>
```


QUIZ

Quick Check: Content Models

Given `<!ELEMENT jobs (president, (manager | clerk)*)>`, which children of `<jobs>` are **valid**?

A. `manager`, `president`

B. `president`, `clerk`, `manager`, `clerk`

C. no children: `<jobs/>`

D. `president`, `president`

ANSWER

Quick Check: Content Models

Given `<!ELEMENT jobs (president, (manager | clerk)*)>`, which children of `<jobs>` are **valid**?

- A `manager`, `president`
- B `president`, `clerk`, `manager`, `clerk`
- C no children: `<jobs/>`
- D `president`, `president`

Why

One `president` must come first; after it, any number of `manager` and `clerk` elements in any mix. A has the wrong order, C lacks the president, D has two.

6 · VALIDATING XML WITH A DTD

Declaring Attributes

employees.dtd

```
<!ELEMENT employee (employee_id, last_name)>
<!ATTLIST employee
    manager_id  CDATA             #IMPLIED
    status      (active|retired)  "active"
    badge       ID                #REQUIRED>
```

employee (valid)

```
<employee manager_id="102" badge="E104">
  <employee_id>104</employee_id>
  <last_name>Ernst</last_name>
</employee>
```

Type	Value must be
`CDATA`	any text
`(a|b|c)`	one of the listed tokens
`ID`	a name unique in the document
`IDREF(S)`	the ID(s) of other elements
`NMTOKEN(S)`	name token(s): no spaces
`ENTITY`, `NOTATION`	declared entity / notation names

Attribute Defaults

Default	Meaning
`#REQUIRED`	The document must specify the attribute
`#IMPLIED`	Optional; no default value
`"value"`	Optional; the parser supplies `value` when it is missing
`#FIXED "value"`	Optional, but if present it must equal `value`; always reported

```
<!ATTLIST employee department_id (10|60|90)  "90">
<!ATTLIST employee manager_id    CDATA       #IMPLIED>
<!ATTLIST employee min_salary    CDATA       #FIXED "4000">
<!ATTLIST employee badge         ID          #REQUIRED>
```

Key idea

A validating parser *adds* defaults: `<employee badge="E1"/>` is reported with `department_id="90"` and `min_salary="4000"`.

General Entities

DTD

```
<!ENTITY company "Bahçeşehir University">
<!ENTITY motto   "Learn &amp; build">
<!ENTITY signature "<sig>HR Department</sig>">
<!ENTITY footer  SYSTEM "footer.xml">
```

document

```
<letter>&company; — &motto; &signature;</letter>
```

Kind	Declared	Referenced
Predefined	by XML (5)	`<` `&` …
Character	by Unicode	`©`
Internal general	`"text"`	`&name;`
External parsed	`SYSTEM "uri"`	`&name;`
Parameter	`% name`	`%name;` (DTD only)

Watch out

External entities can read local files — the basis of the **XXE** attack (Section 8).

Parameter Entities and Notations

hr.dtd (external file)

```
<!ENTITY % person "(id, first_name, last_name)">
<!ELEMENT employee %person;>
<!ELEMENT manager  %person;>
```

- Declared with `% name` (note the space)
- Referenced as `%name;` — only inside DTDs
- Reuse a content model in one place
- Internal subset: only *between* declarations

notations

```
<!NOTATION jpeg SYSTEM "image/jpeg">
<!NOTATION gif  SYSTEM "image/gif">
<!ENTITY logo SYSTEM "logo.jpg" NDATA jpeg>
<!ATTLIST image
    source CDATA              #REQUIRED
    type   NOTATION (gif|jpeg) #REQUIRED>
```

Note

Notations and unparsed (`NDATA`) entities are rarely used today: binary data is referenced by URL or embedded as Base64.

A Complete DTD Example

employees.dtd

```
<!ELEMENT employees (employee+)>
<!ELEMENT employee (employee_id, last_name)>
<!ATTLIST employee
    manager_id    CDATA      #IMPLIED
    department_id (10|60|90) "90">
<!ELEMENT employee_id (#PCDATA)>
<!ELEMENT last_name (#PCDATA)>
```

employees.xml

```
<?xml version="1.0"?>
<!DOCTYPE employees SYSTEM "employees.dtd" [
  <!ENTITY title "Mr">
]>
<employees>
  <employee manager_id="100"
            department_id="10">
    <employee_id>101</employee_id>
    <last_name>&title; Kochhar</last_name>
  </employee>
</employees>
```

Validating Against a DTD

TERMINAL

```
$ xmllint --noout --valid employees.xml
$ xmllint --noout --dtdvalid employees.dtd staff.xml
staff.xml:3: element employee: validity error :
  Element employee content does not follow the DTD,
  expecting (employee_id , last_name), got (last_name )
```

Tool	How
`xmllint` (libxml2)	`--valid` uses the document's DOCTYPE; `--dtdvalid` a given file
Python + `lxml`	`etree.DTD(...).validate(doc)` — code in Section 8
IDEs	VS Code XML extension, PyCharm: validate while typing

Watch out

Python's standard library parsers (`expat`) are **non-validating**: they check well-formedness only.

TRY IT

Write a DTD for a Course Catalogue

catalogue.xml

```xml
<catalogue>
  <course code="AIN3005" level="bachelor">
    <title>Advanced Python Programming</title>
    <instructor>Binnur Kurt</instructor>
    <prerequisite>AIN2001</prerequisite>
  </course>
</catalogue>
```

- At least one `course`
- A course: `title`, one or more `instructor`s, any number of `prerequisite`s — in this order
- `code` required and unique in the document
- `ects` optional, default `6`
- `level`: `bachelor` or `master`, optional

ANSWER

Write a DTD for a Course Catalogue

catalogue.dtd

```
<!ELEMENT catalogue (course+)>
<!ELEMENT course (title, instructor+, prerequisite*)>
<!ATTLIST course
    code   ID                 #REQUIRED
    ects   CDATA              "6"
    level  (bachelor|master)  #IMPLIED>
<!ELEMENT title (#PCDATA)>
<!ELEMENT instructor (#PCDATA)>
<!ELEMENT prerequisite (#PCDATA)>
```

Key idea

`ID` enforces uniqueness — two courses with `code="AIN3005"` make the document invalid ("ID AIN3005 already defined").

07

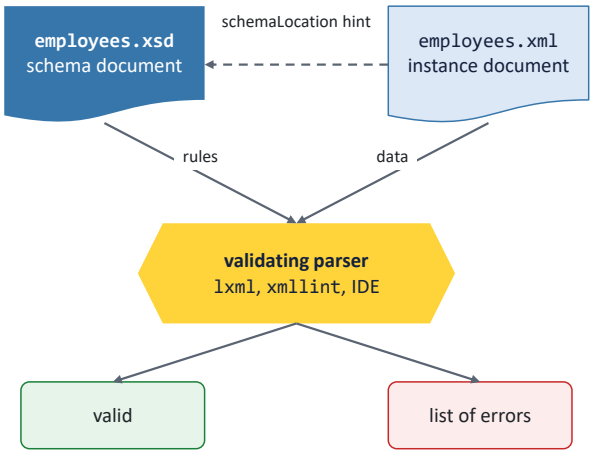
Validating XML with XML Schema

XSD: typed, namespace-aware schemas written in XML

- Schema documents, instance documents and namespaces
- Built-in types and simple types: restriction, list, union
- Complex types: sequence, choice, all and groups
- Attributes, wildcards, documentation; XSD versus DTD

What Is XML Schema?

- **XML Schema** (XSD): a W3C language for describing XML documents
- A schema is itself XML, stored in an `.xsd` file
- A document checked against it is an **instance document**
- Adds **data types**: `xs:int`, `xs:date`, ranges, patterns
- Namespace-aware; reusable named types and groups
- Used by UBL e-invoices, SOAP/WSDL, Office Open XML



A First Schema

region.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="region">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="region_id" type="xs:int"/>
        <xs:element name="region_name" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Note

`xs:` is only a prefix (`xsd:` is just as common); the namespace URI is what counts.

region.xml (instance document)

```xml
<region xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="region.xsd">
  <region_id>1</region_id>
  <region_name>Europe</region_name>
</region>
```

The schema Element and Target Namespaces

departments.xsd

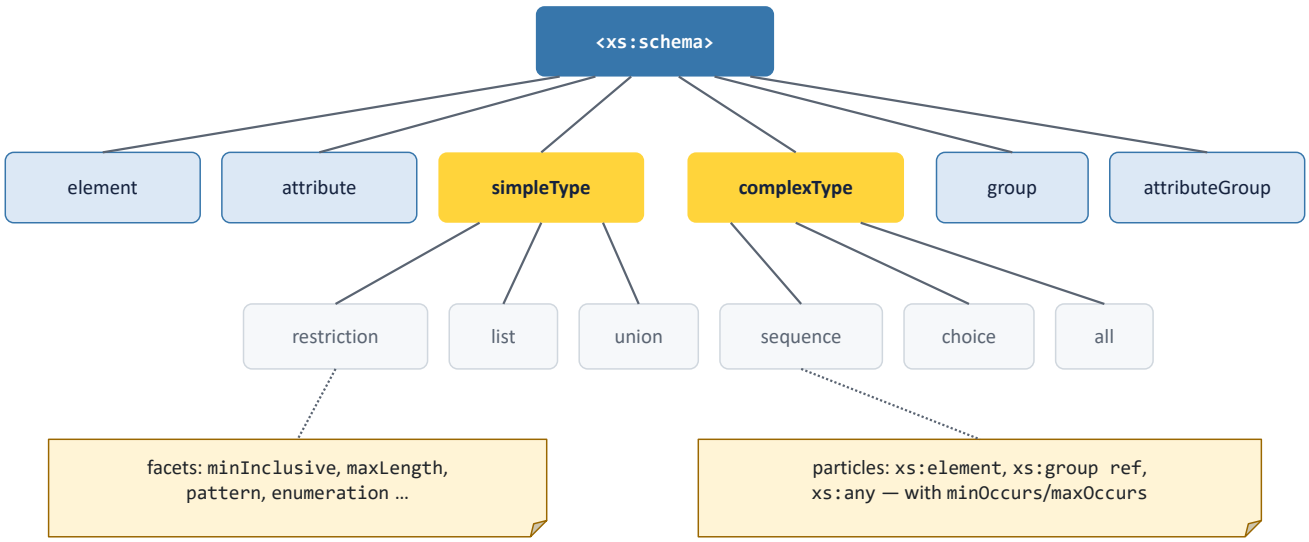
```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="http://example.com/hr"
           xmlns="http://example.com/hr"
           elementFormDefault="qualified" version="1.0">
  <xs:element name="departments" type="xs:string"/>
</xs:schema>
```

departments.xml

```xml
<departments xmlns="http://example.com/hr"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://example.com/hr departments.xsd">
  Finance
</departments>
```

Attribute	Purpose
`targetNamespace`	Namespace of the elements this schema declares
`elementFormDefault="qualified"`	Local elements belong to the target namespace too
`xsi:schemaLocation`	Pairs: *namespace URI ␣ schema location*

Schema Building Blocks



Global and Local Declarations

```
1  <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
2             targetNamespace="http://example.com/hr"
3             xmlns:hr="http://example.com/hr"
4             elementFormDefault="qualified">
5    <xs:element name="email" type="xs:string"/>
6    <xs:complexType name="contactType">
7      <xs:sequence>
8        <xs:element ref="hr:email"/>
9        <xs:element name="phone" type="xs:string"/>
10     </xs:sequence>
11   </xs:complexType>
12   <xs:element name="contact" type="hr:contactType"/>
13 </xs:schema>
```

- **Global**: direct child of `xs:schema` (lines 5, 6, 12)
  - Can be referenced: `ref=`, `type=`
  - Global elements may be the root
- **Local**: nested inside another component (line 9)
  - Visible only in that context
- References need the target-namespace prefix: `hr:`

Declaring Elements

```
<xs:sequence>
  <xs:element name="first_name" type="xs:string"/>
  <xs:element name="phone" type="xs:string" minOccurs="0" maxOccurs="3"/>
  <xs:element name="country" type="xs:string" default="Türkiye"/>
  <xs:element name="currency" type="xs:string" fixed="TRY"/>
</xs:sequence>
```

Attribute	Meaning
`name` / `type`	Tag name; built-in or named type of the content
`ref`	Reuse a global element instead of declaring a new one
`minOccurs` / `maxOccurs`	Occurrence range (default 1..1; `unbounded` = no limit)
`default` / `fixed`	Value used for an empty element / the only allowed value

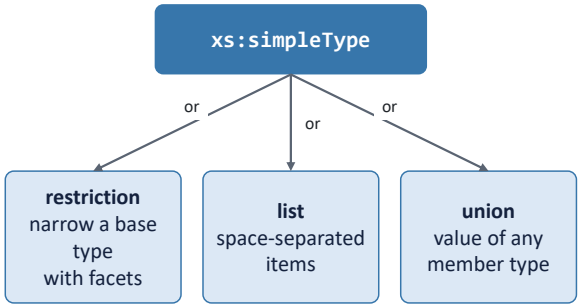
> **Watch out**
> `minOccurs`/`maxOccurs` belong on *local* declarations (inside a sequence, choice or all), not on global ones.

Built-in Data Types

Family	Types	Example values
Strings	`string`, `normalizedString`, `token`	`Ada Lovelace`
Exact numbers	`decimal`, `integer`, `int`, `long`, `positiveInteger`	`1000.50`, `42`
Floating point	`float`, `double`	`6.02E23`, `INF`, `NaN`
Logical	`boolean`	`true`, `false`, `1`, `0`
Date and time	`date`, `time`, `dateTime`, `duration`, `gYear`	`2026-09-27`, `P1Y2M`
Identifiers	`ID`, `IDREF`, `NMTOKEN`, `Name`, `language`	`E104`, `tr-TR`
Other	`anyURI`, `base64Binary`, `hexBinary`, `QName`	`https://bau.edu.tr`

> **Note**
> All 44 built-in types derive from `anySimpleType` in a hierarchy — e.g. `byte` ⊂ `short` ⊂ `int` ⊂ `long` ⊂ `integer` ⊂ `decimal`.

Simple Types: Deriving New Types




```xml
<xs:simpleType name="empIdType">
  <xs:restriction base="xs:positiveInteger">
    <xs:maxInclusive value="1000"/>
  </xs:restriction>
</xs:simpleType>

<xs:element name="employee_id" type="empIdType"/>
```

Instance	Valid?
`<employee_id>999</employee_id>`	yes
`<employee_id>0</employee_id>`	no: not positive
`<employee_id>1001</employee_id>`	no: above 1000

Restricting Values: Ranges and Enumerations

range [0, 100]

```
<xs:simpleType name="ageType">
  <xs:restriction base="xs:integer">
    <xs:minInclusive value="0"/>
    <xs:maxInclusive value="100"/>
  </xs:restriction>
</xs:simpleType>
```

- `<age>42</age>` valid
- `<age>101</age>`, `<age>-1</age>` invalid

fixed list of values

```
<xs:simpleType name="brandType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Audi"/>
    <xs:enumeration value="BMW"/>
    <xs:enumeration value="Volkswagen"/>
  </xs:restriction>
</xs:simpleType>
```

- `<brand>BMW</brand>` valid
- `<brand>bmw</brand>` invalid: case-sensitive

Restricting Strings: Length and Patterns

length

```
<xs:simpleType name="passwordType">
  <xs:restriction base="xs:string">
    <xs:minLength value="8"/>
    <xs:maxLength value="64"/>
  </xs:restriction>
</xs:simpleType>
```

digits

```
<xs:simpleType name="priceType">
  <xs:restriction base="xs:decimal">
    <xs:minInclusive value="0"/>
    <xs:fractionDigits value="2"/>
  </xs:restriction>
</xs:simpleType>
```

pattern

```
<xs:simpleType name="courseCodeType">
  <xs:restriction base="xs:string">
    <xs:pattern value="[A-Z]{3}[0-9]{4}"/>
  </xs:restriction>
</xs:simpleType>
```

- `AIN3005` valid; `ain3005`, `AIN305` invalid
- Patterns are anchored: the *whole* value must match
- Several `pattern` facets = alternatives (OR)

Facets Reference

Facet	Constrains	Example
`enumeration`	value must be one of a list	`<xs:enumeration value="Monday"/>`
`length`	exact length (characters or list items)	`<xs:length value="5"/>`
`minLength` / `maxLength`	minimum / maximum length	`<xs:maxLength value="10"/>`
`pattern`	regular expression over the text	`<xs:pattern value="[A-Z]{3}"/>`
`minInclusive` / `maxInclusive`	range, bounds included	`<xs:minInclusive value="18"/>`
`minExclusive` / `maxExclusive`	range, bounds excluded	`<xs:maxExclusive value="100.0"/>`
`totalDigits`	maximum number of digits	`<xs:totalDigits value="5"/>`
`fractionDigits`	maximum digits after the point	`<xs:fractionDigits value="2"/>`
`whiteSpace`	`preserve`, `replace` or `collapse`	`<xs:whiteSpace value="collapse"/>`

List Types

```xml
<xs:simpleType name="lotteryNumberType">
  <xs:restriction base="xs:integer">
    <xs:minInclusive value="1"/>
    <xs:maxInclusive value="49"/>
  </xs:restriction>
</xs:simpleType>

<xs:simpleType name="lotteryTicketType">
  <xs:list itemType="lotteryNumberType"/>
</xs:simpleType>

<xs:element name="lottery" type="lotteryTicketType"/>
```

instance

```xml
<lottery>4 8 15 16 23 42</lottery>
```

- Items separated by white space
- `itemType`: an atomic type (or a union) — no lists of lists
- Limit the count: restrict the list type with `xs:length`

Common misconception

A list of `xs:string` splits at spaces: `Ada Lovelace` is **two** items.

Union Types

member types

```
<xs:simpleType name="sizeByNumber">
  <xs:restriction base="xs:positiveInteger">
    <xs:maxInclusive value="42"/>
  </xs:restriction>
</xs:simpleType>

<xs:simpleType name="sizeByName">
  <xs:restriction base="xs:string">
    <xs:enumeration value="small"/>
    <xs:enumeration value="medium"/>
    <xs:enumeration value="large"/>
  </xs:restriction>
</xs:simpleType>
```

union

```
<xs:element name="jeansSize">
  <xs:simpleType>
    <xs:union memberTypes="sizeByNumber
                           sizeByName"/>
  </xs:simpleType>
</xs:element>
```

both valid

```
<jeansSize>36</jeansSize>
<jeansSize>small</jeansSize>
```

Note

Members are tried in order; the first type that accepts the value wins.

TRY IT

Define a Grade Type

- `letterGrade`: one of `AA BA BB CB CC DC DD FD FF`
- `gpaType`: a decimal from `0.00` to `4.00` with at most two decimal places
- `gradeOrGpa`: accepts a value of either type
- Then declare an element `grade` of type `gradeOrGpa`

Tip

Hint: enumeration, minInclusive/maxInclusive, fractionDigits and union are all you need.

ANSWER

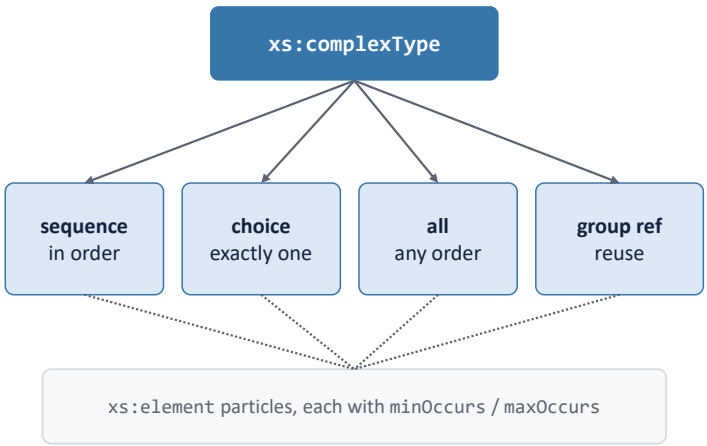
Define a Grade Type

```
<xs:simpleType name="letterGrade">
  <xs:restriction base="xs:string">
    <xs:enumeration value="AA"/>
    <xs:enumeration value="BA"/>
    <!-- BB CB CC DC DD FD likewise -->
    <xs:enumeration value="FF"/>
  </xs:restriction>
</xs:simpleType>
```

```
<xs:simpleType name="gpaType">
  <xs:restriction base="xs:decimal">
    <xs:minInclusive value="0"/>
    <xs:maxInclusive value="4"/>
    <xs:fractionDigits value="2"/>
  </xs:restriction>
</xs:simpleType>
<xs:simpleType name="gradeOrGpa">
  <xs:union memberTypes="letterGrade gpaType"/>
</xs:simpleType>
<xs:element name="grade" type="gradeOrGpa"/>
```


7 · VALIDATING XML WITH XML SCHEMA

Complex Types



- Needed for child elements *or* attributes
- **Named** (global, reusable) or **anonymous** (inline)
- Content: one model group + attributes after it
- `mixed="true"`: text between the children
- Empty content: no model group at all
- `xs:simpleContent`: text plus attributes

sequence: Children in Order

```xml
<xs:element name="department">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="dept_id" type="xs:int"/>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="manager_id" type="xs:int"
                  minOccurs="0"/>
      <xs:element name="location_id" type="xs:int"
                  minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

valid: optional children omitted

```xml
<department>
  <dept_id>10</dept_id>
  <name>Administration</name>
</department>
```

invalid: wrong order

```xml
<department>
  <dept_id>20</dept_id>
  <name>Marketing</name>
  <location_id>1800</location_id>
  <manager_id>201</manager_id>
</department>
```


choice: Exactly One Alternative

```xml
<xs:complexType name="contractType">
  <xs:choice>
    <xs:element name="full_time" type="xs:string"/>
    <xs:element name="part_time" type="xs:string"/>
  </xs:choice>
</xs:complexType>

<xs:element name="employee">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="full_name" type="xs:string"/>
      <xs:element name="contract" type="contractType"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

valid

```xml
<employee>
  <full_name>Steven King</full_name>
  <contract>
    <full_time>Permanent</full_time>
  </contract>
</employee>
```

invalid: both alternatives

```xml
<contract>
  <full_time>Permanent</full_time>
  <part_time>Six months</part_time>
</contract>
```

all and Named Model Groups

all: any order

```
<xs:complexType name="addressType">
  <xs:all>
    <xs:element name="street" type="xs:string"/>
    <xs:element name="city" type="xs:string"/>
    <xs:element name="zip" type="xs:string"
                minOccurs="0"/>
  </xs:all>
</xs:complexType>
```

- Each child at most once (XSD 1.0)
- `xs:all` must be the whole content model

group: define once, reference many times

```
<xs:group name="personGroup">
  <xs:sequence>
    <xs:element name="firstName" type="xs:string"/>
    <xs:element name="lastName" type="xs:string"/>
    <xs:element name="birthDate" type="xs:date"/>
  </xs:sequence>
</xs:group>

<xs:complexType name="personInformation">
  <xs:sequence>
    <xs:group ref="personGroup"/>
    <xs:element name="country" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```


QUIZ

Quick Check: Occurrence Constraints

A `book` has `<xs:sequence>` of `title`, `author` (`maxOccurs="unbounded"`) and `year` (`minOccurs="0"`). Which child list is **valid**?

A. `title`, `year`

B. `author`, `title`

C. `title`, `author`, `author`

D. `title`, `author`, `year`, `year`

ANSWER

Quick Check: Occurrence Constraints

A book has `<xs:sequence>` of `title`, `author` (`maxOccurs="unbounded"`) and `year` (`minOccurs="0"`). Which child list is **valid**?

A `title, year`

B `author, title`

C `title, author, author`

D `title, author, year, year`

Why

`author` is required (default `minOccurs=1`) and may repeat; `year` is optional but may appear at most once; order is fixed.

7 · VALIDATING XML WITH XML SCHEMA

Declaring Attributes in XSD

```
<xs:complexType name="documentType">
  <xs:attribute name="language" type="xs:language"/>
  <xs:attribute name="currency" type="xs:string"
                use="required"/>
  <xs:attribute name="department_id" type="xs:int"
                default="10"/>
  <xs:attribute name="version" type="xs:string"
                fixed="1.0"/>
</xs:complexType>
```

Attribute	Meaning
`use`	`optional` (default), `required`, `prohibited`
`default`	value when absent (only if optional)
`fixed`	the only allowed value
`ref`	reuse a global attribute declaration

with an anonymous simple type

```
<xs:attribute name="department_id">
  <xs:simpleType>
    <xs:restriction base="xs:positiveInteger">
      <xs:maxInclusive value="100"/>
    </xs:restriction>
  </xs:simpleType>
</xs:attribute>
```

Key idea

Attribute types are always **simple** types. Inside a complex type, attributes come *after* the model group.

Text Content Plus Attributes

```xml
<xs:element name="price">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:decimal">
        <xs:attribute name="currency"
                      type="xs:string"
                      use="required"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

instance

```xml
<price currency="EUR">12.50</price>
```

- A simple type cannot carry attributes
- `xs:simpleContent` + `xs:extension`: take a simple type and add attributes
- Typical for amounts with currencies, measurements with units
- `xs:complexContent` + `xs:extension` extends complex types (type inheritance)

Empty Elements and Attribute Groups

```xml
<xs:attributeGroup name="deptAttributes">
  <xs:attribute name="id" type="xs:int" use="required"/>
  <xs:attribute name="name" type="xs:string"/>
</xs:attributeGroup>
<xs:element name="departments">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="department" maxOccurs="unbounded">
        <xs:complexType>
          <xs:attributeGroup ref="deptAttributes"/>
        </xs:complexType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

- Empty content = complex type **without** a model group
- **Attribute group**: a named, reusable set of attributes
- Referenced with `ref=`; groups may nest (no cycles)

instance: both department forms are empty

```xml
<departments>
  <department id="10" name="Administration"/>
  <department id="20" name="Marketing"></department>
</departments>
```

Wildcards and Documentation

wildcards: open content

```xml
<xs:complexType name="commentType">
  <xs:sequence>
    <xs:element name="author" type="xs:string"/>
    <xs:any namespace="##other" processContents="lax"
            minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:anyAttribute processContents="skip"/>
</xs:complexType>
```

- `namespace`: `##any`, `##other`, `##local`, `##targetNamespace` or URIs
- `processContents`: `strict` (default), `lax`, `skip`

annotation

```xml
<xs:element name="salary" type="xs:decimal">
  <xs:annotation>
    <xs:documentation xml:lang="en">
      Monthly gross salary in TRY.
    </xs:documentation>
    <xs:appinfo>widget=currency</xs:appinfo>
  </xs:annotation>
</xs:element>
```

Tip

Prefer `xs:annotation` to XML comments: tools and documentation generators can read it.

A Complete Schema: employees.xsd

employees.xsd (1/2): named types

```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:simpleType name="deptType">
    <xs:restriction base="xs:int">
      <xs:enumeration value="10"/>
      <xs:enumeration value="60"/>
      <xs:enumeration value="90"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="employeeType">
    <xs:sequence>
      <xs:element name="employee_id"
                  type="xs:positiveInteger"/>
      <xs:element name="last_name" type="xs:string"/>
    </xs:sequence>
    <xs:attribute name="manager_id"
                  type="xs:positiveInteger"/>
    <xs:attribute name="department_id"
                  type="deptType" default="90"/>
  </xs:complexType>
```

employees.xsd (2/2): root element

```xml
  <xs:element name="employees">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="employee" type="employeeType"
                    maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

DTD	XML Schema
`(10|60|90) "90"`	enumeration type + `default`
`(employee_id, last_name)`	`xs:sequence`
`#PCDATA`	typed: `xs:positiveInteger`
`employee+`	`maxOccurs="unbounded"`

XML Schema Versus DTD

Feature	DTD	XML Schema
Syntax	Own syntax, inherited from SGML	XML
Data types	Text only (plus ID, enumerations)	44 built-in + user-defined types
Namespaces	Not supported	Full support (`targetNamespace`)
Occurrences	`?`, `*`, `+`	Any `minOccurs` / `maxOccurs`
Reuse	Parameter entities	Named types, groups, extension/restriction
Entities	Yes (general and parameter)	No
Location	Inside the document or external	Separate `.xsd` file
Python validation	`lxml`: `etree.DTD`	`lxml`: `etree.XMLSchema`

From DTD to Schema

document

```xml
<employee>
  <employee_id>120</employee_id>
  <last_name>Weiss</last_name>
</employee>
```

DTD: everything is text

```
<!ELEMENT employee (employee_id, last_name)>
<!ELEMENT employee_id (#PCDATA)>
<!ELEMENT last_name (#PCDATA)>
```

TERMINAL

```
$ trang employee.dtd employee.xsd
```

XML Schema: typed

```xml
<xs:complexType name="employeeType">
  <xs:sequence>
    <xs:element name="employee_id"
                type="xs:int"/>
    <xs:element name="last_name"
                type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<xs:element name="employee"
            type="employeeType"/>
```

TRY IT

Write a Schema for the Course Catalogue

Rewrite your catalogue DTD from Section 6 as an XML Schema, and add data types:

- `code` and every `prerequisite`: three capital letters + four digits
- `ects`: a positive integer, default `6`
- `title` and `instructor`: strings; at least one `instructor`
- Use named types: `courseCode` and `courseType`

Tip

Start from the structure: which elements are complex, which are simple?

ANSWER

Write a Schema for the Course Catalogue

catalogue.xsd (1/2)

```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:simpleType name="courseCode">
    <xs:restriction base="xs:string">
      <xs:pattern value="[A-Z]{3}[0-9]{4}"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:element name="catalogue">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="course" type="courseType"
                    maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
```

catalogue.xsd (2/2)

```xml
  <xs:complexType name="courseType">
    <xs:sequence>
      <xs:element name="title" type="xs:string"/>
      <xs:element name="instructor" type="xs:string"
                  maxOccurs="unbounded"/>
      <xs:element name="prerequisite"
                  type="courseCode"
                  minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute name="code" type="courseCode"
                  use="required"/>
    <xs:attribute name="ects" type="xs:positiveInteger"
                  default="6"/>
  </xs:complexType>
</xs:schema>
```

08

Parsing XML in Python

SAX, DOM, ElementTree and lxml — and parsing untrusted XML safely

- Event-based parsing with xml.sax
- Tree-based parsing with xml.dom.minidom
- xml.etree.ElementTree: the everyday API
- Validation with lxml; security with defusedxml

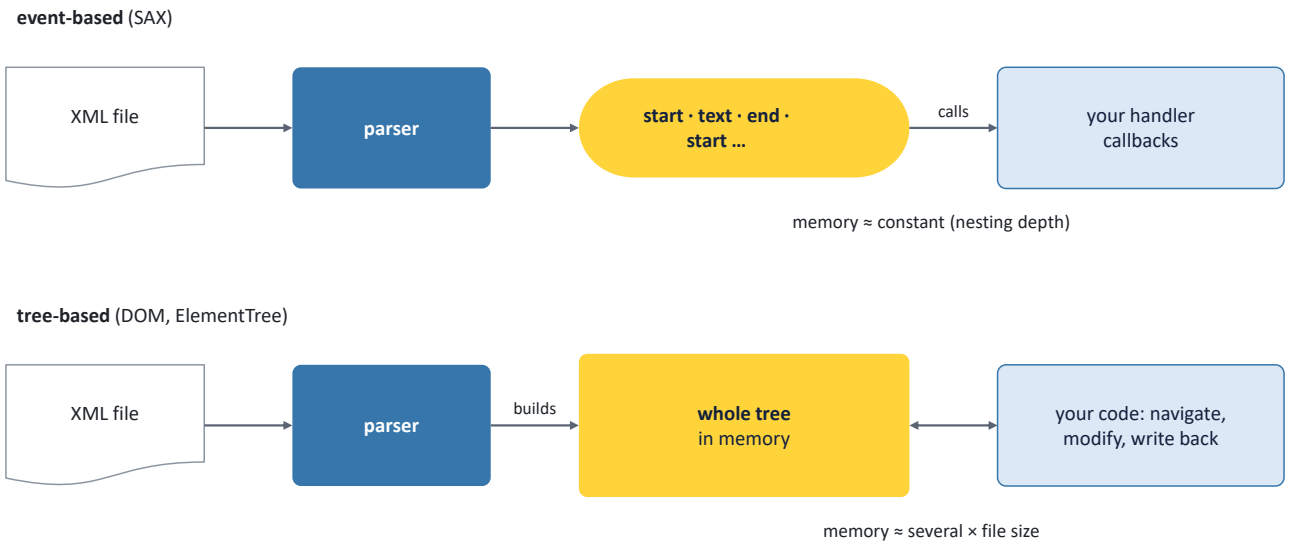
Python's XML Toolbox

Module	Model	Typical use
`xml.sax`	Events pushed to your handler	Huge files, single pass
`xml.dom.minidom`	W3C DOM tree	Small documents, code ported from JS/Java
`xml.etree.ElementTree`	Pythonic tree + `iterparse`	Everyday reading and writing
`xml.dom.pulldom`, `XMLPullParser`	Pull events on demand	Streaming with partial trees
`lxml.etree` (third party)	ElementTree API on libxml2	Full XPath, XSLT, validation
`defusedxml` (third party)	Hardened drop-in parsers	XML from untrusted sources

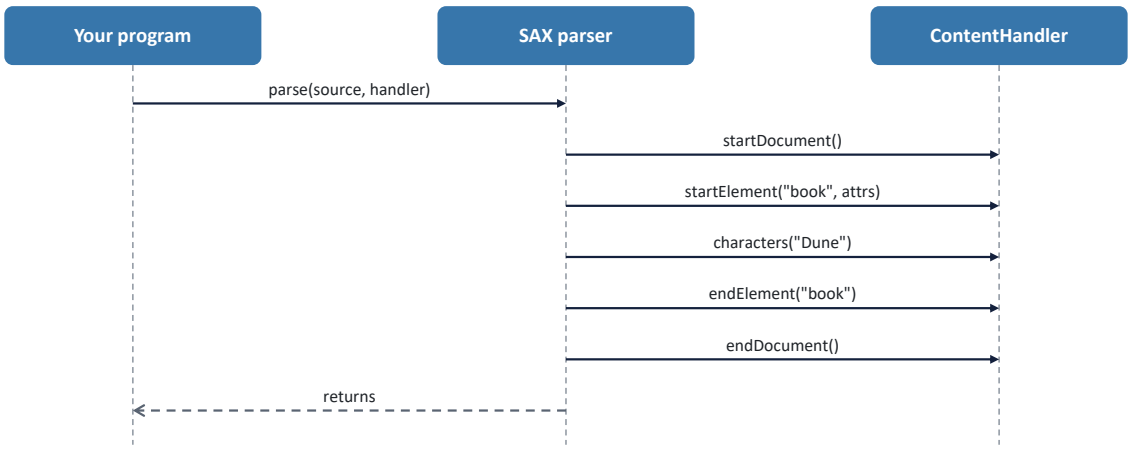
Key idea

All standard-library parsers are built on **expat**: fast, well-formedness checking, *non-validating*.

Event-Based Versus Tree-Based Parsing



SAX: Callbacks for Parsing Events



Key idea

You never call the handler methods — the parser calls *you* (inversion of control).

SAX: Watching the Events

sax_events.py

```python
import xml.sax


class EventPrinter(xml.sax.ContentHandler):
    """Print every event the SAX parser reports."""

    def startElement(self, name, attrs):
        print("start", name, dict(attrs))

    def characters(self, content):
        if content.strip():
            print("text ", repr(content))

    def endElement(self, name):
        print("end  ", name)


xml.sax.parseString(b'<book id="7"><title>Dune</title></book>',
                    EventPrinter())
```

OUTPUT

```
start book {'id': '7'}
start title {}
text  'Dune'
end   title
end   book
```


SAX Case Study: The Data

countries.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<countries>
  <country>
    <Code>NLD</Code>
    <Name>Netherlands</Name>
    <Continent>Europe</Continent>
    <Region>Western Europe</Region>
    <Population>15864000</Population>
    <SurfaceArea>41526.00</SurfaceArea>
  </country>
  <!-- TUR and JPN follow ... -->
</countries>
```

Goal: stream the file and build a `list[Country]`

country.py

```python
from dataclasses import dataclass


@dataclass
class Country:
    code: str = ""
    name: str = ""
    continent: str = ""
    population: int = 0
    surface_area: float = 0.0


FIELDS = {  # XML tag -> (attribute, converter)
    "Code": ("code", str),
    "Name": ("name", str),
    "Continent": ("continent", str),
    "Population": ("population", int),
    "SurfaceArea": ("surface_area", float),
}
```

SAX Case Study: The Content Handler

country_handler.py

```python
import xml.sax

from country import FIELDS, Country


class CountryHandler(xml.sax.ContentHandler):
    def __init__(self) -> None:
        super().__init__()
        self.countries: list[Country] = []
        self._text = ""

    def startElement(self, name, attrs):
        self._text = ""
        if name == "country":
            self.countries.append(Country())

    def characters(self, content):
        self._text += content          # may arrive in several chunks

    def endElement(self, name):
        if name in FIELDS:
            attribute, convert = FIELDS[name]
            setattr(self.countries[-1], attribute,
                    convert(self._text.strip()))
```


SAX Case Study: Running the Parser

main.py

```python
import xml.sax

from country_handler import CountryHandler

parser = xml.sax.make_parser()
handler = CountryHandler()
parser.setContentHandler(handler)
parser.parse("countries.xml")

for country in handler.countries:
    density = country.population / country.surface_area
    print(f"{country.code} {country.name:<12} {country.continent:<7}"
          f"{country.population:>12,} {density:7.1f}")
```

OUTPUT

```
NLD Netherlands  Europe   15,864,000   382.0
TUR Turkey       Asia     66,591,000    85.9
JPN Japan        Asia    126,714,000   335.4
```

FIND THE BUG

Why Is the Country Name Cut Off?

```
import xml.sax


class NameHandler(xml.sax.ContentHandler):
    def startDocument(self):
        self.names, self.in_name = [], False

    def startElement(self, name, attrs):
        self.in_name = name == "Name"

    def characters(self, content):
        if self.in_name:
            self.names.append(content)

    def endElement(self, name):
        self.in_name = False


handler = NameHandler()
xml.sax.parseString(b"<countries><Name>Bosnia &amp;amp; Herzegovina</Name>"
                    b"<Name>Trinidad and Tobago</Name></countries>", handler)
for text in handler.names:
    print(repr(text))
```

OUTPUT

```
'Bosnia '
'&'
' Herzegovina'
'Trinidad and Tobago'
```


ANSWER

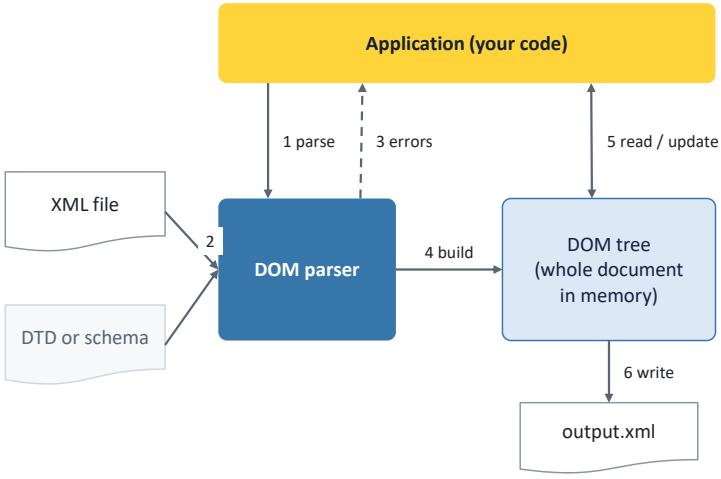
Why Is the Country Name Cut Off?

- `characters()` may be called **several times** for one text node
- expat splits text at entity references, buffer boundaries and line ends
- Fix: collect chunks between start and end tag, join them in `endElement`

```
def characters(self, content):
    if self.in_name:
        self.chunks.append(content)

def endElement(self, name):
    if name == "Name":
        self.names.append("".join(self.chunks))
        self.chunks = []
    self.in_name = False
```

DOM: The Whole Document as a Tree



- **DOM** (Document Object Model): W3C standard tree API
- Level 1 (1998), Level 2 (2000, namespaces), Level 3 (2004)
- Same interfaces in JavaScript, Java, Python (`xml.dom.minidom`)
- Random access, many passes, add / remove / modify nodes
- Cost: the whole document in memory

Reading XML with minidom

dom_read.py

```python
from xml.dom import minidom

XML_TEXT = """<countries>
  <country><Code>NLD</Code><Name>Netherlands</Name>
    <Population>15864000</Population></country>
  <country><Code>JPN</Code><Name>Japan</Name>
    <Population>126714000</Population></country>
</countries>"""


def text_of(parent, tag: str) -> str:
    """Return the text inside the first <tag> below parent."""
    return parent.getElementsByTagName(tag)[0].firstChild.data


document = minidom.parseString(XML_TEXT)   # or minidom.parse(path)
for country in document.getElementsByTagName("country"):
    population = int(text_of(country, "Population"))
    print(text_of(country, "Code"), text_of(country, "Name"),
          f"{population:,}")
```

OUTPUT

```
NLD Netherlands 15,864,000
JPN Japan 126,714,000
```

DOM Nodes and White Space

```
>>> from xml.dom import minidom
>>> doc = minidom.parseString("<book>\n  <title>Dune</title>\n</book>")
>>> book = doc.documentElement
>>> [node.nodeName for node in book.childNodes]
['#text', 'title', '#text']
>>> book.firstChild.data
'\n  '
>>> book.getElementsByTagName("title")[0].firstChild.data
'Dune'
```

- Everything is a node: Document, Element, Text, Comment, Attr
- Indentation between tags = Text nodes
- `node.nodeType == node.ELEMENT_NODE` filters elements
- `getElementsByTagName` skips text nodes

Watch out

`firstChild` of a pretty-printed element is usually white space, not the first child element.

Modifying and Writing a DOM

dom_write.py

```
from xml.dom import minidom

doc = minidom.parseString(
    '<countries><country code="NLD"/></countries>')
root = doc.documentElement

country = doc.createElement("country")    # new element
country.setAttribute("code", "TUR")
name = doc.createElement("Name")
name.appendChild(doc.createTextNode("Türkiye"))
country.appendChild(name)
root.appendChild(country)                 # attach to tree
root.removeChild(root.firstChild)         # drop NLD

with open("countries.xml", "w", encoding="utf-8") as out:
    doc.writexml(out, encoding="utf-8")
print(doc.toprettyxml(indent="  "))
```

OUTPUT

```
<?xml version="1.0" ?>
<countries>
  <country code="TUR">
    <Name>Türkiye</Name>
  </country>
</countries>
```

ElementTree: Elements, Text and Tail

Element
tag: str attrib: dict[str, str] text: str or None tail: str or None children: 0..n Elements
len(e), e[i], for child in e find(), findall(), iter() get(), set(), append()

```
>>> import xml.etree.ElementTree as ET
>>> p = ET.fromstring('<p lang="en">Hi <b>there</b>, you</p>')
>>> p.tag, p.attrib, p.text
('p', {'lang': 'en'}, 'Hi ')
>>> b = p[0]
>>> b.tag, b.text, b.tail
('b', 'there', ', you')
>>> len(p), [child.tag for child in p]
(1, ['b'])
```

Key idea

`text` = text *before the first child*; `tail` = text *after the element's end tag*, stored on that element.

Parsing and Reading with ElementTree

et_read.py

```
import xml.etree.ElementTree as ET
from pathlib import Path

Path("library.xml").write_text(
    '<library>\n'
    '  <book id="b1" genre="sci-fi"><title>Dune</title></book>\n'
    '  <book id="b2" genre="novel"><title>Emma</title></book>\n'
    '</library>\n', encoding="utf-8")

tree = ET.parse("library.xml")      # ElementTree: whole document
root = tree.getroot()               # Element <library>
print(root.tag, len(root), "books")
for book in root:                   # iterate over child elements
    print(book.get("id"), book.attrib["genre"],
          book.find("title").text)
```

OUTPUT

```
library 2 books
b1 sci-fi Dune
b2 novel Emma
```

Searching with find, findall and iter

et_search.py

```python
import xml.etree.ElementTree as ET

root = ET.fromstring(
    '<library>'
    '<book id="b1" genre="sci-fi"><title>Dune</title>'
    '<price>12.50</price></book>'
    '<book id="b2" genre="novel"><title>Emma</title>'
    '<price>7.25</price></book>'
    '<book id="b3" genre="sci-fi"><title>Solaris</title>'
    '<price>10.00</price></book>'
    '</library>')

first = root.find("book")              # first matching child
print(first.get("id"), first.findtext("title"))
print([book.get("id") for book in root.findall("book")])
print([title.text for title in root.iter("title")])
sci_fi = root.findall("book[@genre='sci-fi']")
print(sum(float(b.findtext("price")) for b in sci_fi))
```

OUTPUT

```
b1 Dune
['b1', 'b2', 'b3']
['Dune', 'Emma', 'Solaris']
22.5
```


XPath Syntax in ElementTree

Syntax	Selects	Example on the library	Result
`tag`, `*`	children with that tag / all children	`findall("book")`	3 books
`.//tag`	descendants at any depth	`findall(".//price")`	3 prices
`..`	parent (within the path)	`find(".//title[.='Dune']/..")`	book `b1`
`[@attr='v']`	attribute equals value	`findall("book[@genre='sci-fi']")`	`b1`, `b3`
`[tag='text']`	has a child with that text	`find("book[title='Emma']")`	book `b2`
`[n]`, `[last()]`	position among siblings (from 1)	`find("book[last()]")`	book `b3`

Note

Only a **subset** of XPath 1.0: no functions such as `count()`, no comparisons such as `@year < 1900`, no absolute paths from an element. Use `lxml` for full XPath.

TRACE THE CODE

Text, Tail and Missing Elements

```python
import xml.etree.ElementTree as ET

root = ET.fromstring("<a x='1'>hi<b>one</b>tail<b y='2'>two</b></a>")
print(root.text, len(root))
print([b.text for b in root.findall("b")])
print(root[0].tail, root[1].attrib)
print(root.find("c"), root.findtext("c", "none"))
```


ANSWER

Text, Tail and Missing Elements

OUTPUT

```
hi 2
['one', 'two']
tail {'y': '2'}
None none
```

- `root.text` is only the text *before* the first child: `hi`
- `tail` belongs to the first `b`; attributes are per element
- `find` returns `None` for no match; `findtext` returns the default

Modifying and Writing XML

et_modify.py

```python
import xml.etree.ElementTree as ET

root = ET.fromstring(
    '<library><book id="b1"><price>20.00</price></book>'
    '<book id="b2"><price>8.00</price></book></library>')

for price in root.iter("price"):                # change text
    price.text = f"{float(price.text) * 0.9:.2f}"
root.find("book[@id='b2']").set("sale", "yes")  # attribute
new_book = ET.SubElement(root, "book", id="b3")  # add child
ET.SubElement(new_book, "price").text = "15.00"
root.remove(root.find("book[@id='b1']"))        # delete

ET.indent(root)                                 # pretty-print
ET.ElementTree(root).write("library.xml", encoding="utf-8",
                           xml_declaration=True)
print(ET.tostring(root, encoding="unicode"))
```

OUTPUT

```
<library>
  <book id="b2" sale="yes">
    <price>7.20</price>
  </book>
  <book id="b3">
    <price>15.00</price>
  </book>
</library>
```


Building XML from Python Data

csv_to_xml.py

```python
import xml.etree.ElementTree as ET

countries = [                       # rows read from world.csv
    ("tur", "turkey", "asia", 80_000_000),
    ("fra", "france", "europe", 67_000_000),
]

root = ET.Element("countries")
for code, name, continent, population in countries:
    country = ET.SubElement(root, "country", code=code)
    ET.SubElement(country, "name").text = name.title()
    ET.SubElement(country, "continent").text = continent
    ET.SubElement(country, "population").text = str(population)

ET.indent(root)
print(ET.tostring(root, encoding="unicode"))
```

OUTPUT

```
<countries>
  <country code="tur">
    <name>Turkey</name>
    <continent>asia</continent>
    <population>80000000</population>
  </country>
  <country code="fra">
    <name>France</name>
    <continent>europe</continent>
    <population>67000000</population>
  </country>
</countries>
```

Namespaces in ElementTree

et_namespaces.py

```python
import xml.etree.ElementTree as ET

root = ET.fromstring("""<shop xmlns="urn:example:shop"
                         xmlns:inv="urn:example:invoice">
  <item sku="A1"><inv:price>12.5</inv:price></item>
</shop>""")

print(root.tag)                                  # Clark notation
NS = {"s": "urn:example:shop", "inv": "urn:example:invoice"}
item = root.find("s:item", NS)
print(item.get("sku"), item.findtext("inv:price", namespaces=NS))
print(root.find("item"))                         # no namespace
print(root.find("{urn:example:shop}item").get("sku"))
```

OUTPUT

```
{urn:example:shop}shop
A1 12.5
None
A1
```


Streaming Large Files with iterparse

et_iterparse.py

```python
import xml.etree.ElementTree as ET
from pathlib import Path

with Path("readings.xml").open("w", encoding="utf-8") as out:
    out.write("<readings>\n")
    for i in range(100_000):
        out.write(f'  <reading sensor="s{i % 3}" value="{i % 50}"/>\n')
    out.write("</readings>\n")

count = total = 0
for event, elem in ET.iterparse("readings.xml", events=("end",)):
    if elem.tag == "reading":
        count += 1
        total += int(elem.get("value"))
        elem.clear()                     # release the element's data
print(count, total / count)
```

OUTPUT

```
100000 24.5
```

Key idea

`iterparse` yields each element when its end tag is read — tree-style access to one record at a time, SAX-like memory use.

lxml: Beyond the Standard Library

TERMINAL

```
$ pip install lxml
```

lxml_xpath.py

```python
from lxml import etree

root = etree.fromstring(
    "<library><book year='1965'>Dune</book>"
    "<book year='1815'>Emma</book></library>")
print(root.xpath("//book[@year < 1900]/text()"))
print(root.xpath("count(//book)"))
print(root.getroottree().getpath(root[1]))
```

OUTPUT

```
['Emma']
2.0
/library/book[2]
```

Feature	ElementTree	lxml
ElementTree API	yes	yes
Full XPath 1.0	subset	yes
XSLT transformations	no	yes
DTD / XSD / RELAX NG validation	no	yes
Broken HTML (`lxml.html`)	no	yes
Installation	standard library	`pip install`

Validating Against a DTD with lxml

validate_dtd.py

```python
from io import StringIO

from lxml import etree

dtd = etree.DTD(StringIO("""
<!ELEMENT employees (employee+)>
<!ELEMENT employee (employee_id, last_name)>
<!ATTLIST employee department_id (10|60|90) "90">
<!ELEMENT employee_id (#PCDATA)>
<!ELEMENT last_name (#PCDATA)>""""))       # or etree.DTD("employees.dtd")

doc = etree.fromstring(
    '<employees><employee department_id="20">'
    '<last_name>Hunold</last_name></employee></employees>')

if dtd.validate(doc):
    print("valid")
for error in dtd.error_log.filter_from_errors():
    print(f"line {error.line}: {error.message}")
```

OUTPUT

```
line 1: Element employee content does not follow the DTD, expecting (employee_id , last_name), got (last_name)
line 1: Value "20" for attribute department_id of employee is not among the enumerated set
```

Validating Against a Schema with lxml

employees.xml

```xml
<employees>
  <employee manager_id="100"
            department_id="90">
    <employee_id>101</employee_id>
    <last_name>Kochhar</last_name>
  </employee>
  <employee department_id="20">
    <employee_id>abc</employee_id>
    <last_name>Hunold</last_name>
  </employee>
</employees>
```

validate_xsd.py

```python
import textwrap

from lxml import etree

xsd_doc = etree.parse("employees.xsd")
schema = etree.XMLSchema(xsd_doc)
document = etree.parse("employees.xml")

if schema.validate(document):
    print("employees.xml is valid")
for error in schema.error_log:
    message = f"line {error.line}: {error.message}"
    print(textwrap.fill(message, width=88,
                        subsequent_indent="    "))
```

OUTPUT

```
line 7: Element 'employee', attribute 'department_id': [facet 'enumeration'] The value
    '20' is not an element of the set {'10', '60', '90'}.
line 8: Element 'employee_id': 'abc' is not a valid value of the atomic type
    'xs:positiveInteger'.
```


XML Attacks: Billion Laughs and XXE

billion laughs: 10^9 × 'lol' ≈ 3 GB

```xml
<?xml version="1.0"?>
<!DOCTYPE lolz [
  <!ENTITY l0 "lol">
  <!ENTITY l1 "&l0;&l0;&l0;&l0;&l0;&l0;&l0;&l0;&l0;&l0;">
  <!ENTITY l2 "&l1;&l1;&l1;&l1;&l1;&l1;&l1;&l1;&l1;&l1;">
  <!-- ... up to l9: each level ten times the previous -->
]>
<lolz>&l9;</lolz>
```

Entity expansion (DoS): a 1 KB file exhausts memory and CPU

xxe_demo.py

```python
import xml.etree.ElementTree as ET

XXE = """<!DOCTYPE order [
  <!ENTITY secret SYSTEM "file:///etc/passwd">
]>
<order>&secret;</order>"""

try:
    ET.fromstring(XXE)
except ET.ParseError as err:
    print("rejected:", err)
```

XXE (external entity): read files, reach internal URLs

OUTPUT OF XXE_DEMO.PY

```
rejected: undefined entity &secret;: line 4, column 7
```

Parsing Untrusted XML Safely

TERMINAL

```
$ pip install defusedxml
```

safe_parse.py

```python
from defusedxml import DefusedXmlException
from defusedxml.ElementTree import fromstring


def parse_upload(data: bytes):
    """Parse XML from an untrusted source."""
    try:
        return fromstring(data)
    except DefusedXmlException as err:
        raise ValueError(f"rejected: {err!r}") from err
```

Parser	XXE	Billion laughs
stdlib (3.7.1+, expat 2.4.1+)	not expanded	stopped by expat limit
`lxml` 5+	not resolved by default	stopped by libxml2 limit
`defusedxml`	rejected	rejected

Watch out

For untrusted input never enable `resolve_entities`, `load_dtd`, `dtd_validation` or network access; cap the input size; validate after parsing.

Choosing a Parsing API

	SAX	DOM (`minidom`)	ElementTree	lxml
Model	push events	W3C node tree	Pythonic tree	ElementTree API
Memory	constant	whole doc (large)	whole doc; `iterparse` streams	same as ET
Random access, modify	no	yes	yes	yes
XPath	no	no	subset	full 1.0
Validation	no	no	no	DTD, XSD, RELAX NG
Best for	huge files, one pass	code ported from JS/Java	everyday work	XPath, XSLT, validation

Key idea

Default to **ElementTree**; stream with `iterparse` (or SAX) when files are huge; add **lxml** for XPath or validation; use **defusedxml** for untrusted input.

QUIZ

Quick Check: Choosing a Parser

You must total the `<amount>` values in a 20 GB XML log on a laptop with 8 GB of RAM. Which approach works?

- **A** `minidom.parse()` and `getElementsByTagName("amount")`
- **B** `ET.parse()` and `root.iter("amount")`
- **C** `ET.iterparse()` with `elem.clear()` (or a SAX handler)
- **D** Read the file with `readlines()` and use `str.find`

ANSWER

Quick Check: Choosing a Parser

You must total the `<amount>` values in a 20 GB XML log on a laptop with 8 GB of RAM. Which approach works?

- **A** `minidom.parse()` and `getElementsByTagName("amount")`
- **B** `ET.parse()` and `root.iter("amount")`
- **C** `ET.iterparse()` with `elem.clear()` (or a SAX handler)
- **D** Read the file with `readlines()` and use `str.find`

★ **Why**

Only streaming keeps memory flat. Both tree approaches load the whole document; text searching ignores XML syntax (CDATA, comments, entities, line breaks).

Practice Problems

1. Log statistics

Count ERROR lines per day in every `*.log` below a folder (`Path.rglob`, `with open`, `encoding="utf-8"`)

2. Safe config update

Load `settings.json`, merge defaults, write to a temporary file and replace the original

3. CSV to XML

Convert `world.csv` into `countries.xml` with ElementTree and validate it against your own XSD

4. XML to JSON

Turn `library.xml` into a list of dicts saved as JSON; convert it back and compare

5. Streaming statistics

Min, max and mean of `<reading value>` in a 1 GB file with `iterparse` — keep memory flat

6. Safe upload

`load_order(data: bytes)`: parse with `defusedxml`, validate with `lxml`, return a dataclass

Key Takeaways

- ✓ Open files with `with` and an explicit `encoding="utf-8"`; context managers guarantee cleanup
- ✓ Use `pathlib.Path` for paths and directories; `shutil`/`os` for the rest
- ✓ JSON ↔ Python with `dump`/`load`; CSV with `csv` readers and writers and `newline=""`
- ✓ XML must be well-formed; a DTD or an XML Schema makes it *valid*
- ✓ XML Schema adds data types, namespaces and reusable simple and complex types
- ✓ ElementTree by default, `iterparse`/SAX for huge files, `lxml` for XPath and validation
- ✓ Treat XML from outside as hostile: `defusedxml`, no external entities

NEXT UP Module 05 — Thread Programming in Python